UNIVERSIDAD COMPLUTENSE DE MADRID

ntic master
revolucionamos la comunicación

Master Data Science, Big Data & Business Analytics

Programación Python (avanzado)



Javier Domínguez Gómez
GnuPG fingerprint: 94AD 19F4 9005 EEB2 3384 C20F 5BDC C668 D664 8E2B

v 1.3.4

Tabla de contenido

Introducción..... 4
Plataformas de desarrollo.....4
Comprobación del entorno de trabajo adecuado.....5
Instalación de Python 3..... 5
Instalación de PyCharm..... 7
Simple programa de ejemplo..... 11
Uso de Google Colab.....17
Métodos de las colecciones..... 20
Métodos en cadenas de texto.....21
Métodos en listas..... 24
Métodos en diccionarios..... 26
Módulos..... 27
Collections.....27
Datetime..... 29
Random..... 30
Manejo de ficheros..... 33
Creación..... 34
Apertura o lectura..... 35
Modificación..... 36
Pandas..... 37
Importación de Pandas..... 37
Importación y visualización con Matplotlib..... 37
Series.....43
Dataframes..... 43
Lectura de ficheros..... 47
Operaciones con los datos.....49
Selección..... 49
Filtrado..... 50
Ordenación.....51
Tratamiento de valores nulos..... 52
Operaciones con strings..... 54
Contar valores.....55
Generación de gráficos..... 55

Agrupación de datos..... 56
Agrupación por nombre de columna..... 57
Agrupación por campo custom..... 61
Stack/Unstack..... 62
Indexación de los datos..... 69
Resetear los índices..... 71
Ejemplo esperanza de vida frente a renta per cápita por países..... 72
Escritura en ficheros..... 80
NumPy..... 82
Arrays de NumPy vs listas tradicionales..... 82
Ejemplo Índice de Masa Corporal (IMC)..... 83
Arrays de dos dimensiones..... 84
Cálculo estadístico..... 86
Servicio web..... 87
Instalación de librerías necesarias..... 87
Desarrollo de una API Rest..... 90
Extracción de datos de una API..... 92

Introducción

Una vez cursado el módulo de **fundamentos de Python**, el alumno podrá continuar su aprendizaje de un modo más avanzado y en el contexto de Big Data. Para ello realizaremos varios ejercicios en cada sección de modo que el aprendizaje sea gradual y efectivo, pues el alumno comenzará a programar desde el primer momento. Al final del módulo se introducirán algunos conceptos básicos y librerías utilizadas en el tratamiento de datos y cálculo numérico como son Pandas y NumPy, a desarrollar una sencilla API que sea capaz de exponer datos y un programa que sea capaz de ingestar datos de una API.

Plataformas de desarrollo

Para iniciarse por primera vez en Python se suele recomendar utilizar algún entorno de desarrollo como **Anaconda**, que incorpora la aplicación web **Jupiter Notebook** o **Google Collaboration** (**Colab**) disponible en la suite de aplicaciones de Google. También existen editores de texto avanzados o IDE, tales como **PyCharm**, que nos permitirá programar en Python de un modo más avanzado, ya que disponen de varias herramientas que nos servirán para comprobar la sintaxis, detectar posibles errores en el código automáticamente o tener nuestro código ordenado y estructurado en diferentes archivos y carpetas.

Para la realización de este módulo de Programación Python avanzado usaremos **Google Collaboration** (**Colab**), y únicamente al final del módulo el IDE **PyCharm**, de modo que todos los alumnos podrán seguir sin problema los ejemplos descritos en esta documentación, así como las explicaciones del profesor en clase.

Comprobación del entorno de trabajo adecuado

Aunque el alumno va a utilizar principalmente **Google Collaboration** (**Colab**) es recomendable instalar Python de forma nativa en su PC o Mac, y luego instalar **PyCharm**. Esto se utilizará únicamente al final del módulo, pero conviene que se instale con antelación.

Nota: Los siguientes pasos están orientados al sistema operativo Windows. Si tu sistema operativo es Mac el proceso es prácticamente igual, cambia muy poco y se pueden seguir los pasos de una forma muy similar. Si tu sistema es GNU/Linux, probablemente ya sabes instalar Python desde la terminal. Si no, me preguntáis.

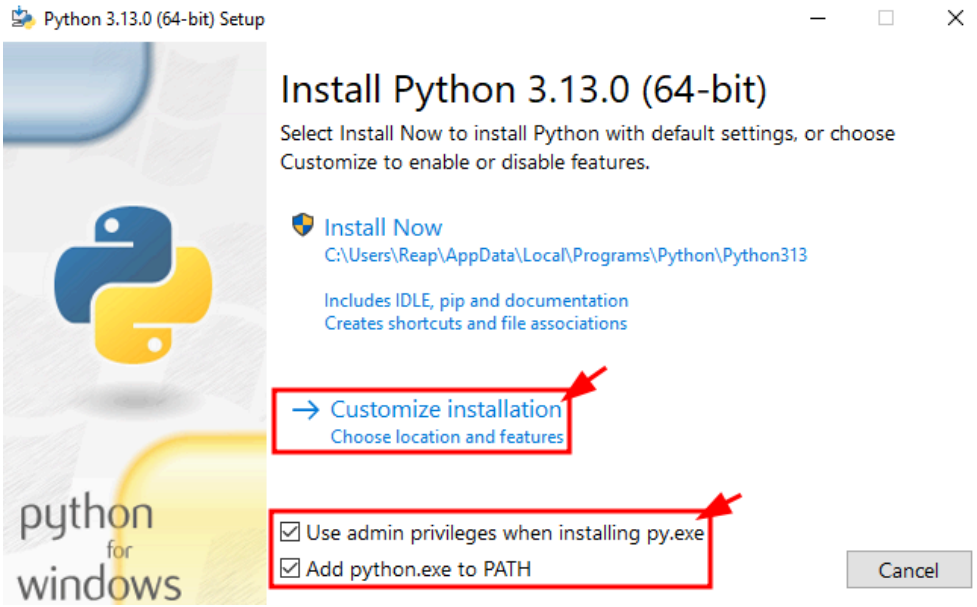
Instalación de Python 3

1. En primer lugar, debemos descargar Python en su versión más reciente desde la página oficial: https://www.python.org/downloads/

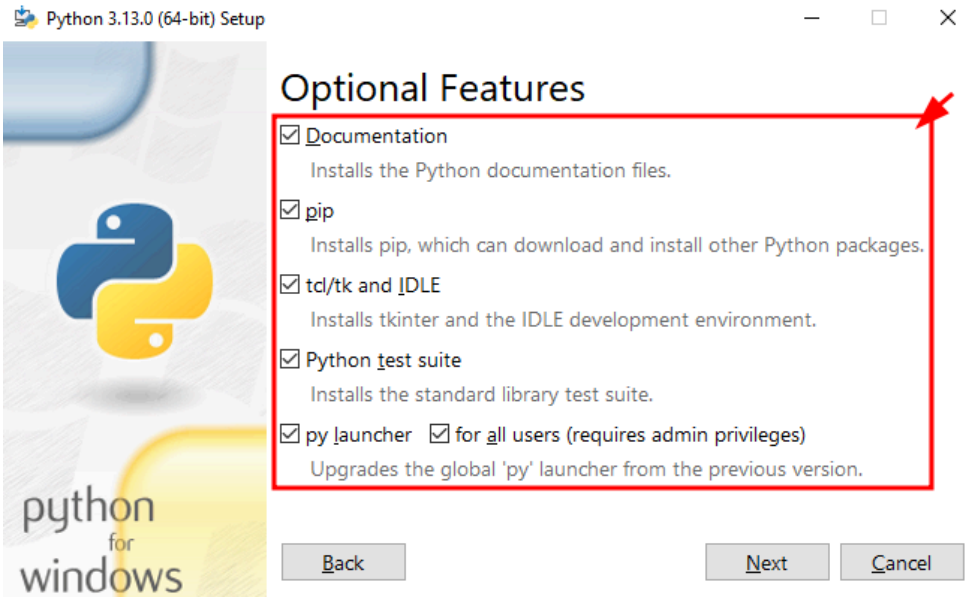
   Nota: Si en el momento de acceder a la página para descargar Python ves que la versión que aparece es superior a la que aparece en esta documentación, no te preocupes. Has de seguir los mismos pasos.



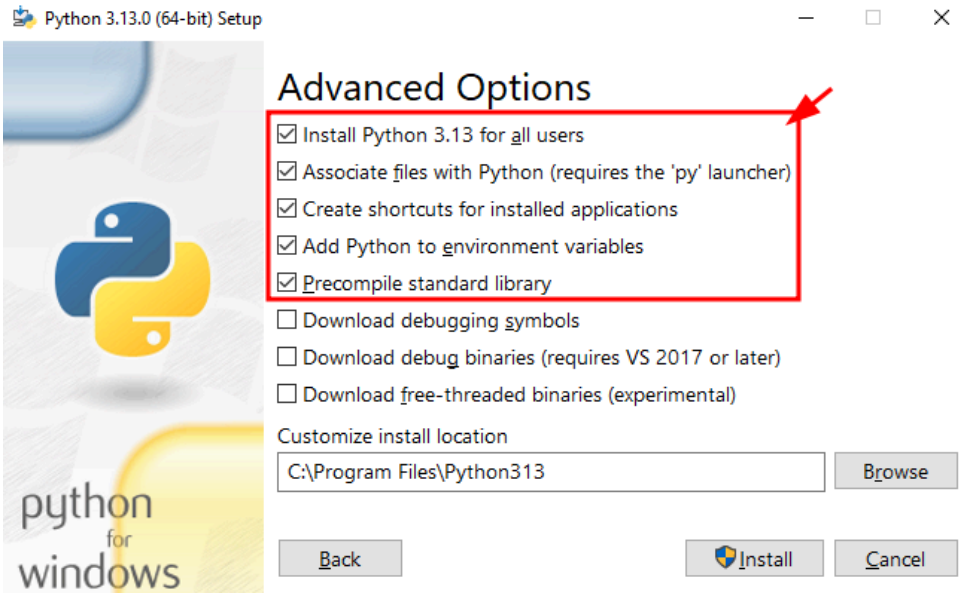
2. Ejecutar el instalador previamente descargado. Hay que asegurarse de tener activados los dos checkbox de abajo, y finalmente seleccionar la opción "*Customize installation*".



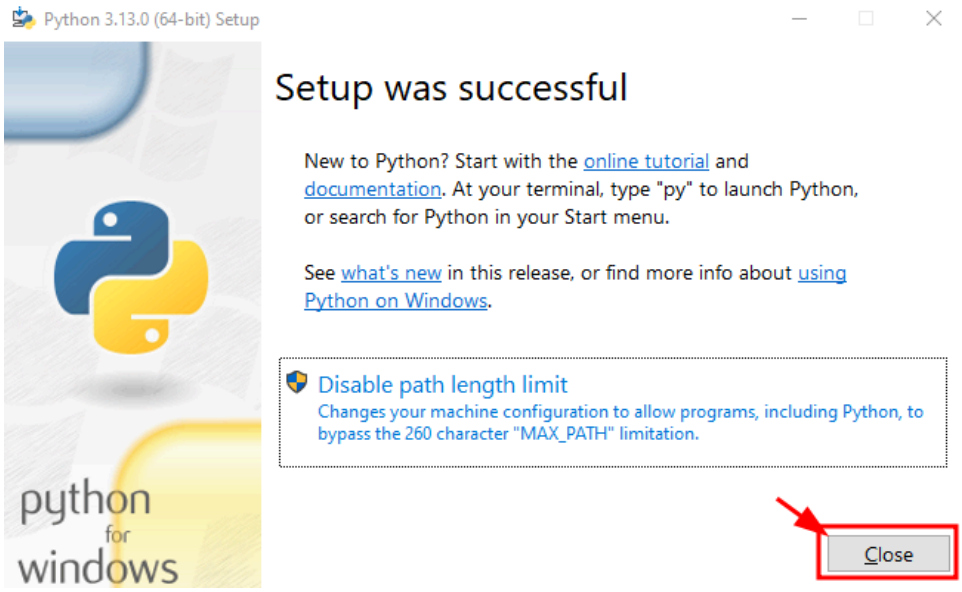
3. En esta pantalla seleccionaremos todos los checkbox y pulsamos el botón "*Next*".



4. En esta pantalla basta con seleccionar todos los checkbox indicados en la siguiente imagen y pulsar el botón "*Install*".

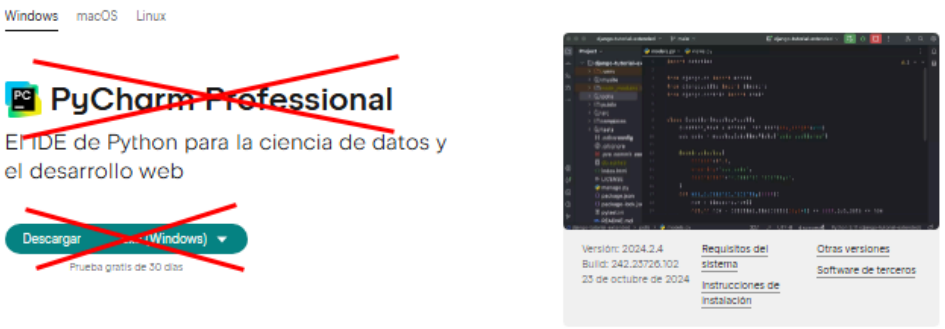


5. La instalación puede demorar unos minutos. Cuando esta finalice veremos una pantalla similar a la siguiente imagen. Pulsamos el botón "*Close*".



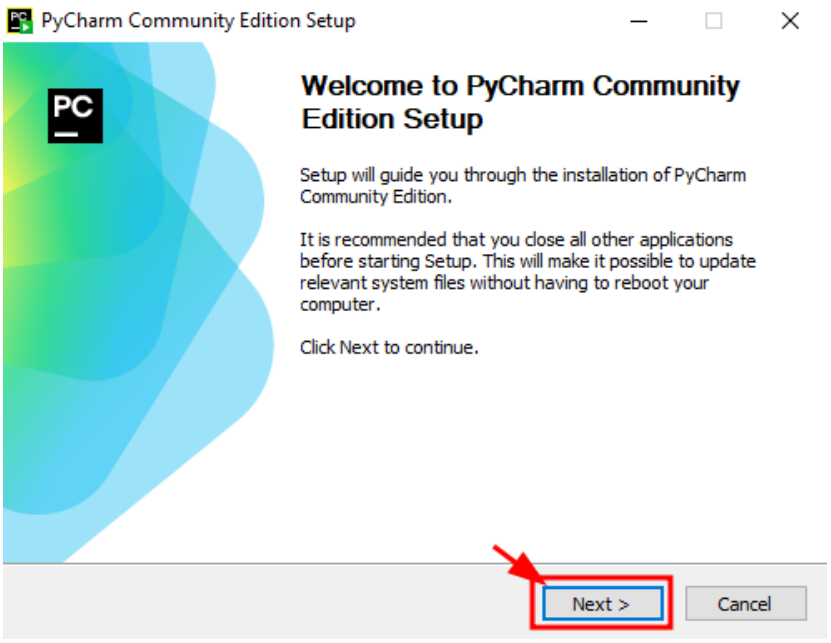
Instalación de PyCharm

1. Debemos descargar PyCharm **Community Edition** (**no descarguéis la Professional**) en su versión más reciente desde la página oficial: https://www.jetbrains.com/es-es/pycharm/download/



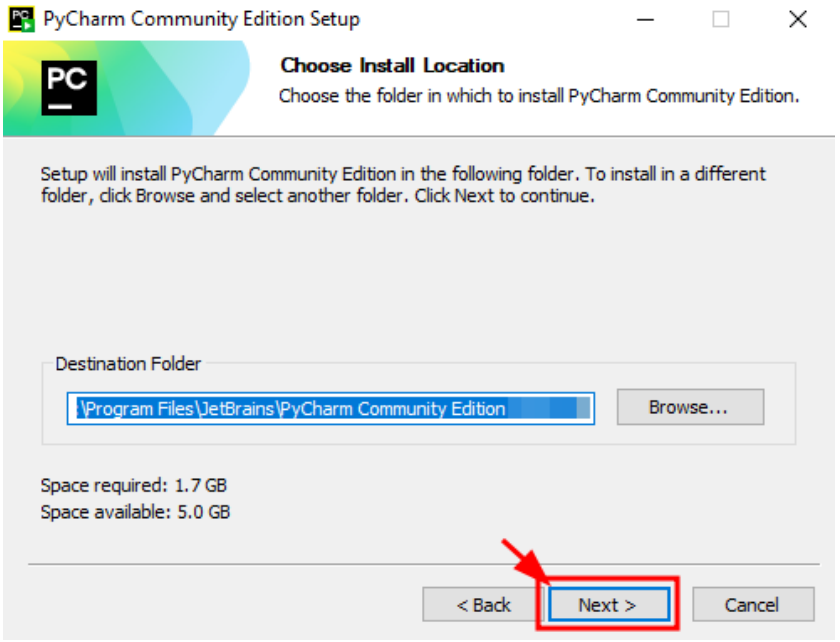


2. Ejecutar el instalador previamente descargado y pulsar el botón "*Next*".

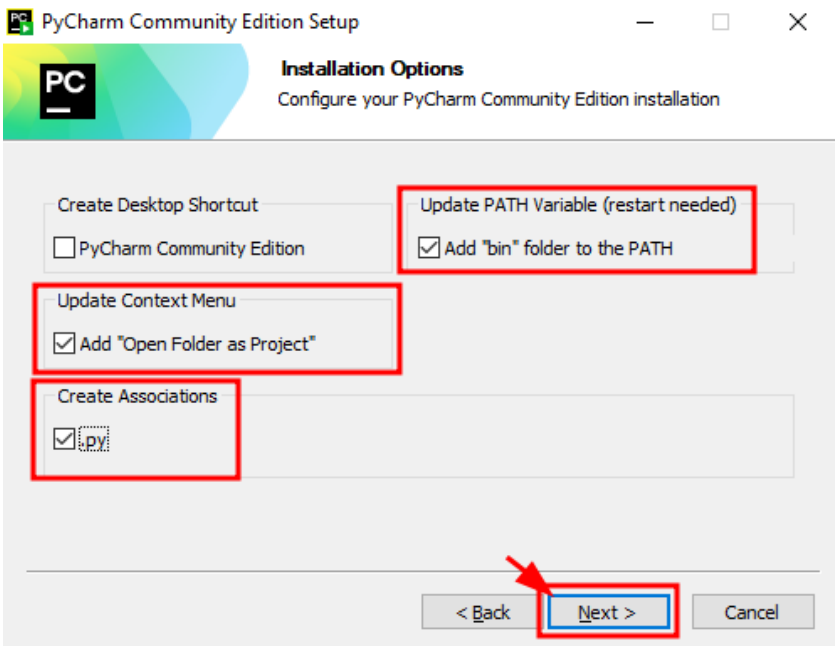


3. Podemos seleccionar la ruta de instalación que queramos, pero recomiendo dejar la ruta que viene por defecto, así será más fácil

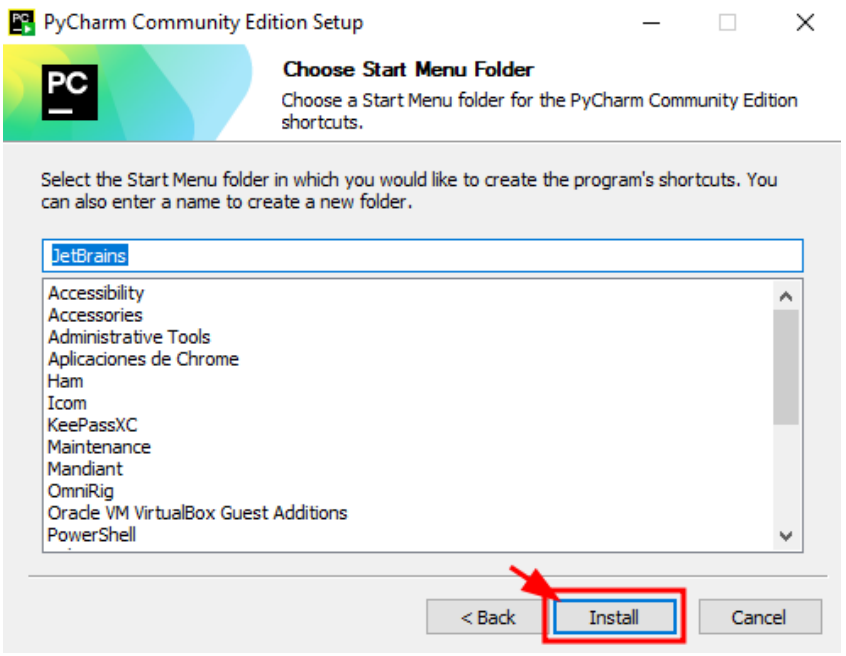
localizar las carpetas de instalación en caso de duda o problemas. Pulsamos el botón "*Next*".



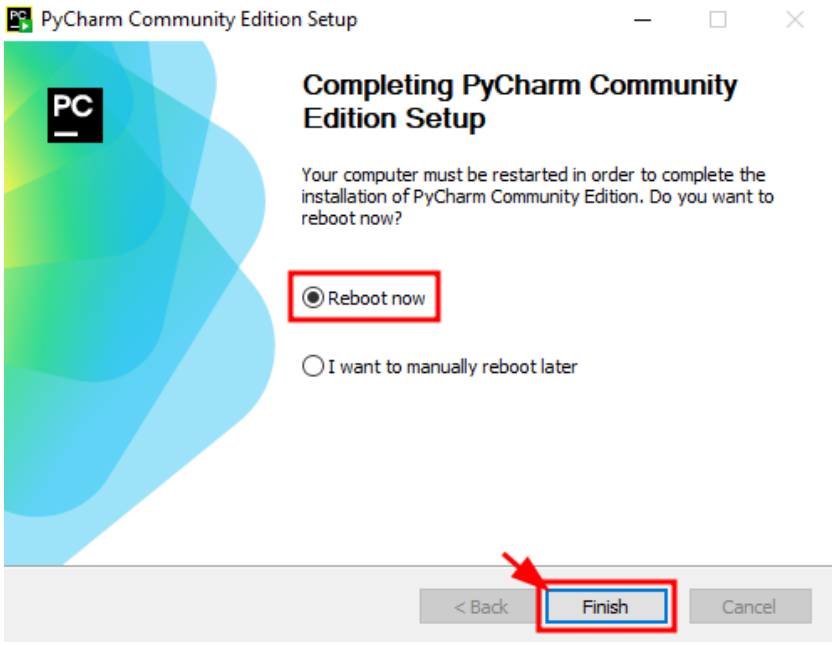
4. Basta con activar los checkbox que se indican en la siguiente imagen y pulsamos el botón "*Next*".



5. Pulsamos el botón "*Install*".

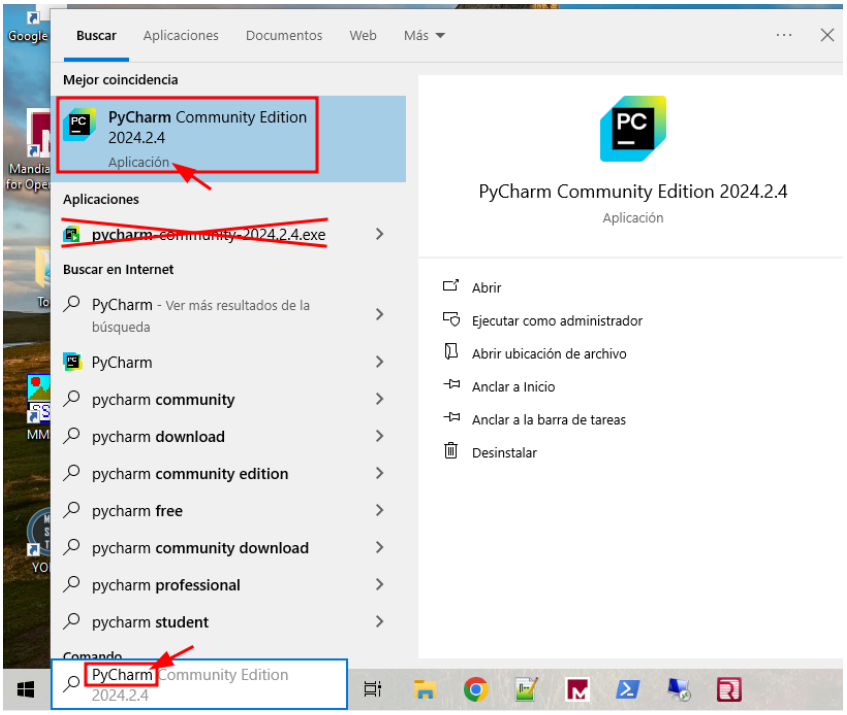


6. La instalación puede demorar unos minutos. Cuando esta finalice veremos una pantalla similar a la siguiente imagen. Seleccionamos la opción "*Reboot now*" y pulsamos el botón "*Finish*".



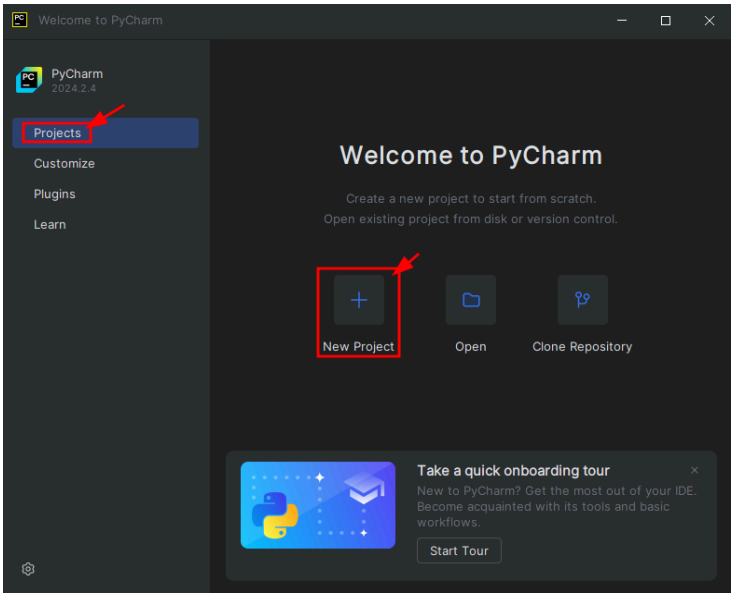
7. Una vez reiniciado el PC, si vamos a la barra de búsqueda de Windows y escribimos "*PyCharm Comunity Edition*" veremos que nos aparece como aplicación instalada. Pulsamos sobre ella (**no sobre el archivo**

ejecutable .exe que nos hemos descargado al principio), tal y como se muestra en la siguiente imagen.



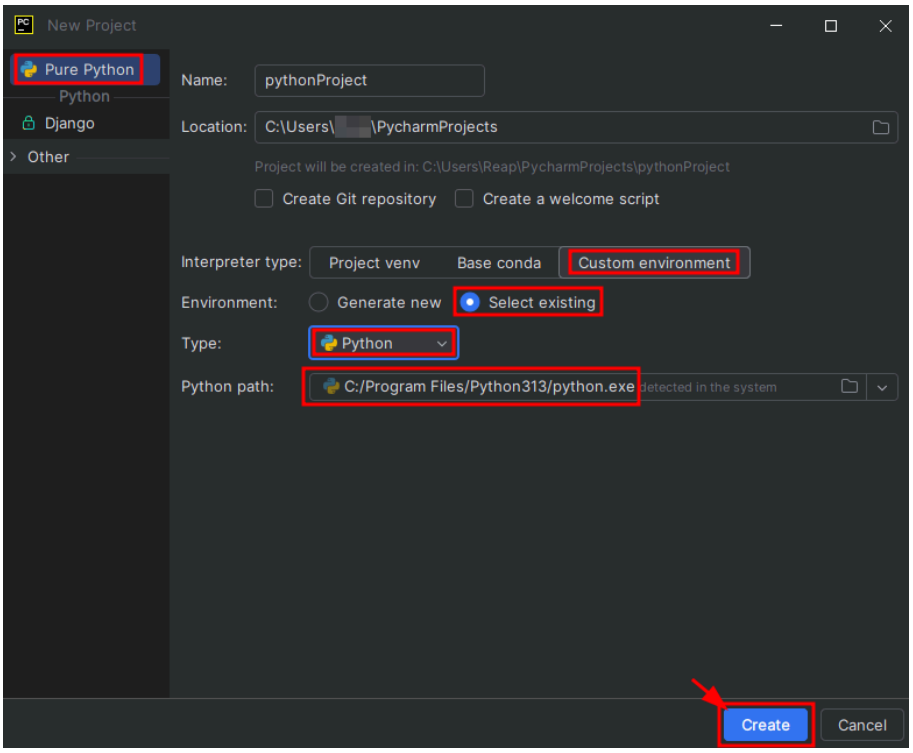
Simple programa de ejemplo

1. Una vez que tenemos abierto PyCharm, seleccionamos en el menú de la izquierda la opción "*Projects*" y luego pulsamos sobre "*New Project*"



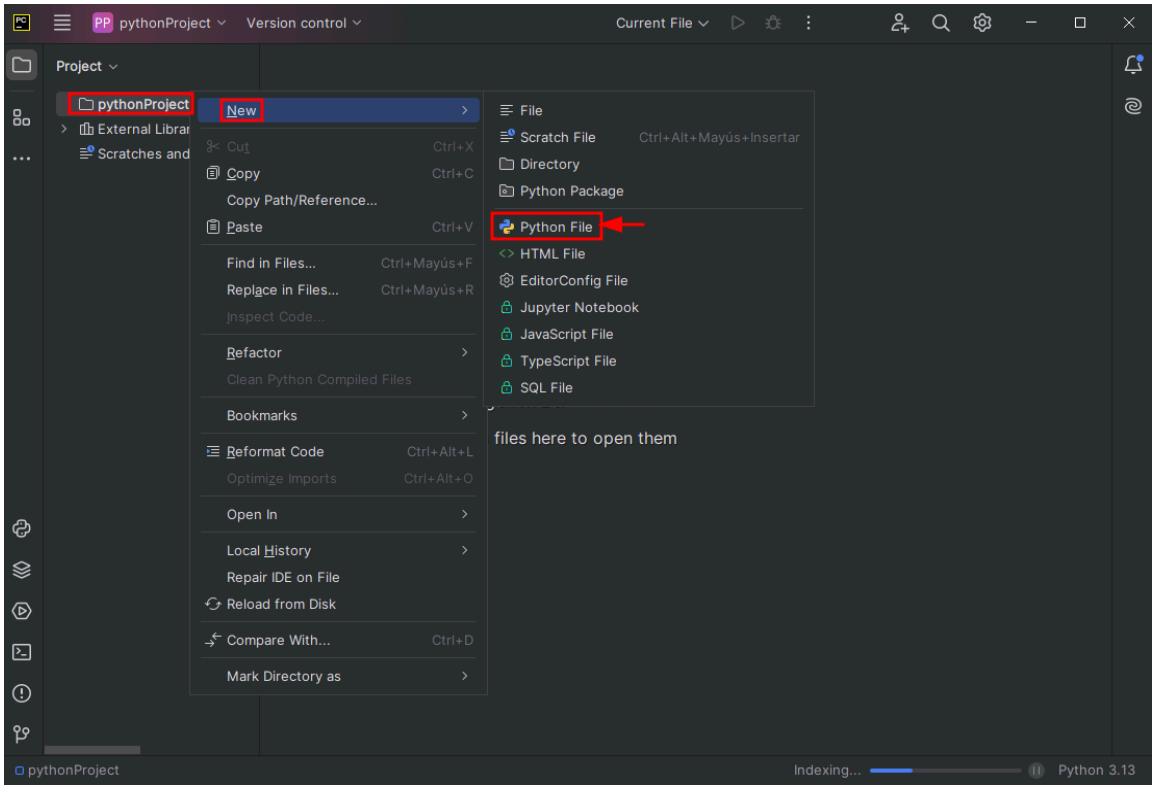
2. En el menú de la izquierda debemos seleccionar la opción "*Pure Python*". En "*Interpreter type*" hay que seleccionar "*Custom environment*", en "*Environment*" seleccionamos "*Select existing*", en "*Type*" seleccionamos "*Python*" y finalmente en "*Python path*" debemos seleccionar la ruta donde tenemos instalado Python en nuestro PC, en este ejemplo es "*C:\Program Files\Python313\python.exe*" (dependiendo de la versión de Python que tengamos instalada en el PC puede variar un poco la ruta).

   Una vez configurado nuestro primer proyecto pulsamos el botón "*Create*".

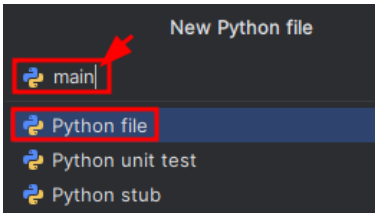


3. En este momento se empezarán a descargar e indexar todos los paquetes de Python. Dependiendo de la velocidad de vuestra conexión a internet esto puede tardar más o menos.

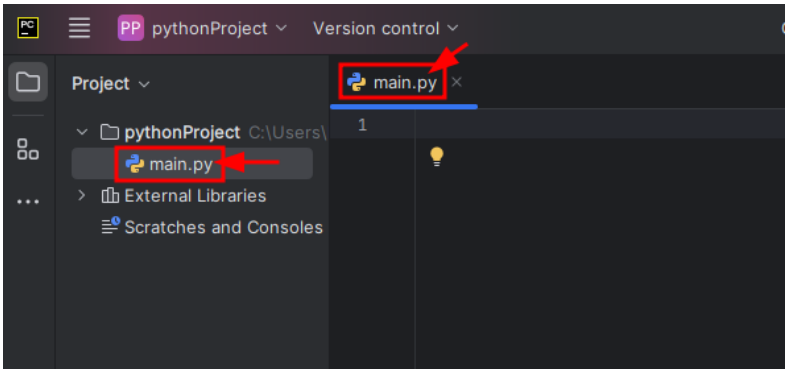
4. Ahora ya podemos crear nuestro primer archivo "*main.py*" en el que iremos escribiendo código Python. Para ello haremos click con el botón derecho del ratón sobre la carpeta de nuestro proyecto "*pyhonProject*", seleccionamos la opción "*New*" y luego "*Python File*".



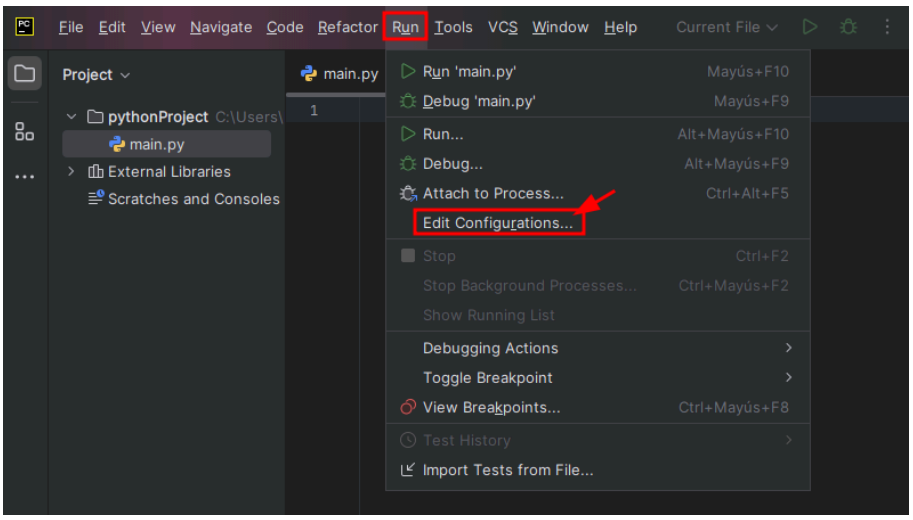
5. Como nombre le ponemos simplemente "*main*" y pulsamos en "*Python file*".



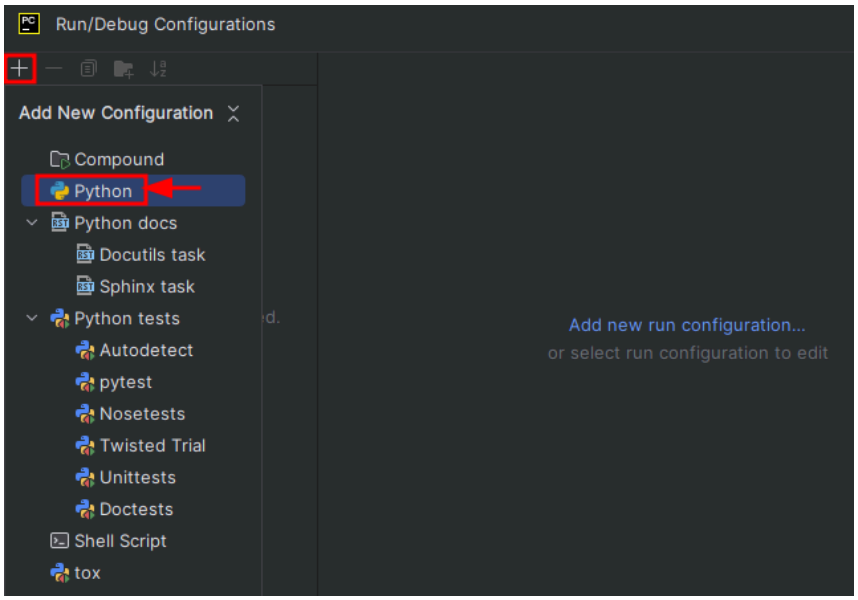
6. Esto nos creará nuestro primer archivo Python en el que ya podremos ir escribiendo código. Hacemos doble click sobre el archivo main.py



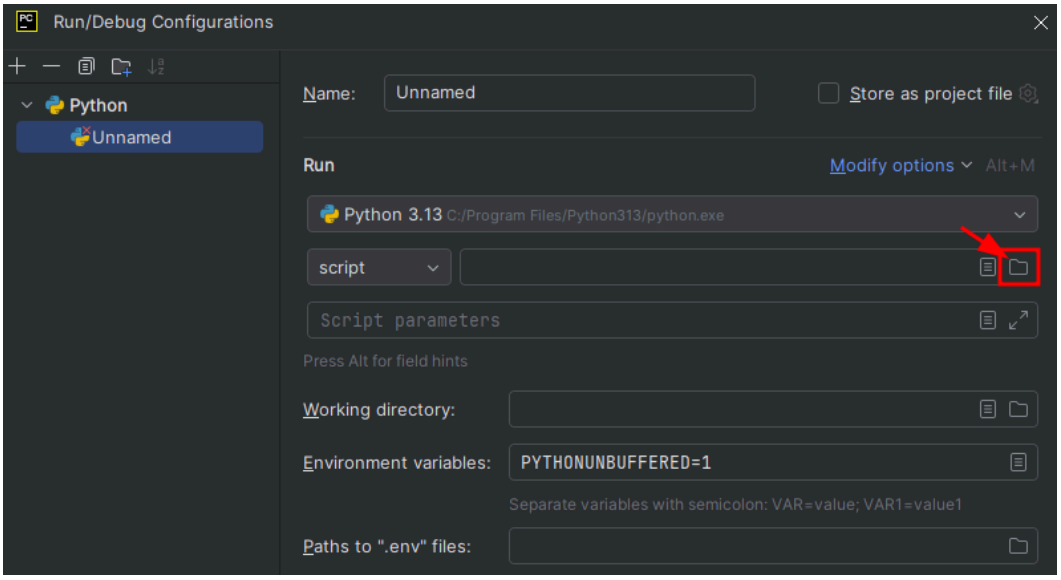
7. En la esquina superior izquierda veremos un icono con tres rayas verticales, al pulsarlo se mostrará el menú superior, vamos a seleccionar la opción "*Run*" y vamos a ver cómo podemos configurar algunas cosas en el proyecto seleccionando la opción "*Edit Configurations…*" del menú, tal y como se muestra en la siguiente imagen.



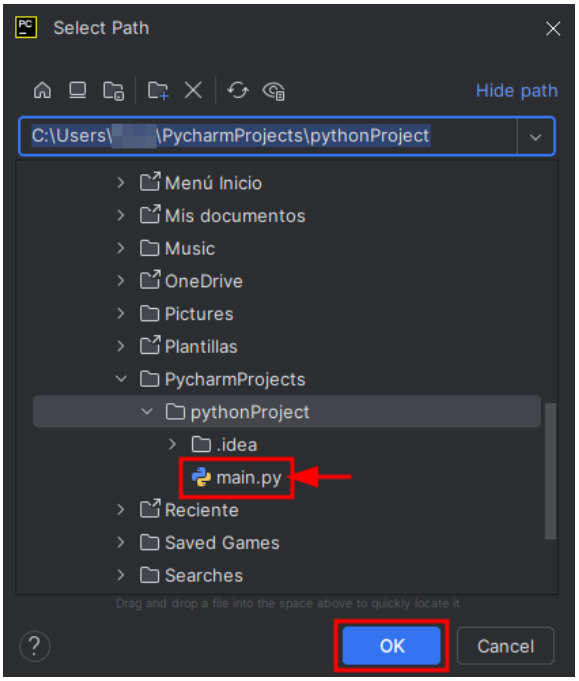
8. Pulsamos en el botón "+" y seleccionamos "*Python*".



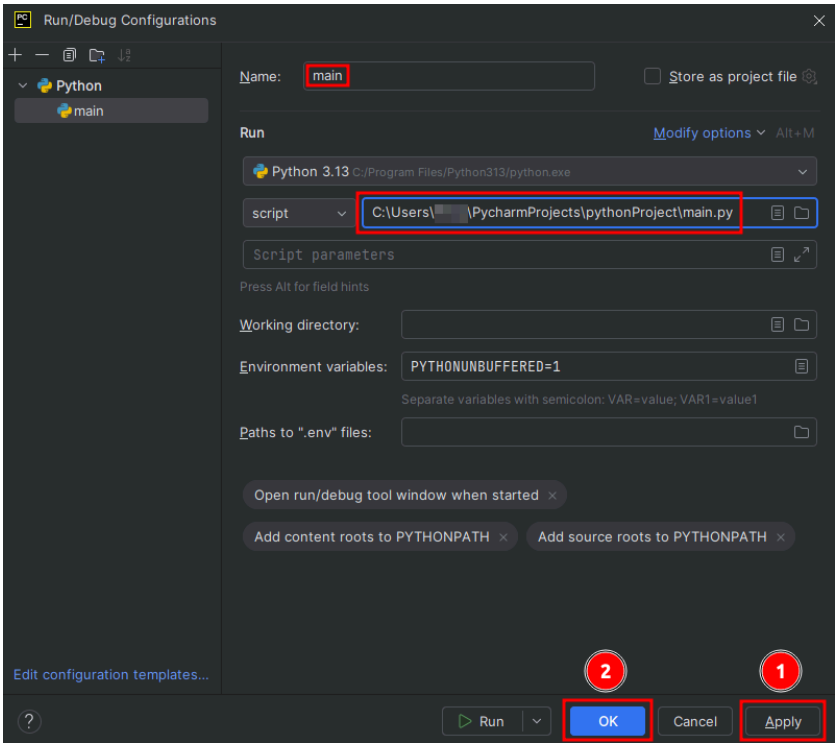
9. En esta ventana, pulsamos únicamente en el icono pequeño con una carpeta que aparece a la derecha del combo "*script*", para seleccionar el archivo "*main.py*" que acabamos de crear en el punto anterior.



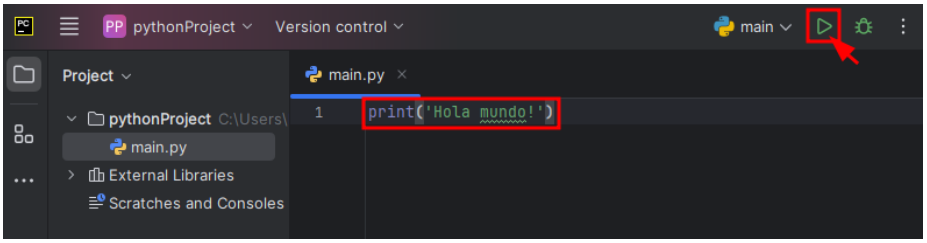
10. Seleccionamos el archivo "*main.py*" de la carpeta de nuestro proyecto.



11. Nos cambiará automáticamente el nombre del campo "*name*" a "*main*". Simplemente pulsamos primero el botón "*Apply*" y luego "*OK*".



12. Ahora escribiremos el programa más sencillo posible, que no es más que una línea **print('hola mundo')**. Para ejecutarlo y comprobar que todo nuestro entorno de programación en Python es adecuado pulsaremos el botón que tiene una forma de triángulo verde ("*Run*") tal y como se muestra en la siguiente imagen.



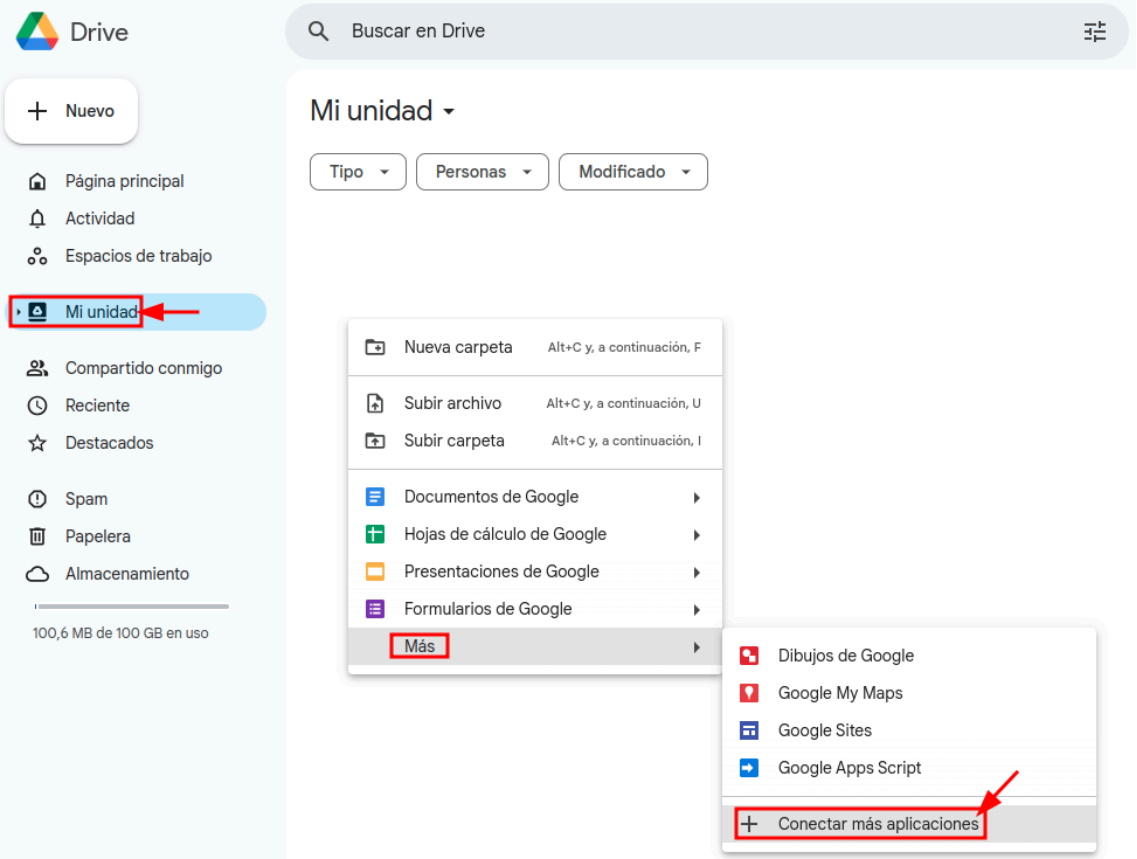
13. En la consola de la parte inferior veremos el resultado de la ejecución de este script, y debe mostrar un mensaje que diga "*Hola mundo*".



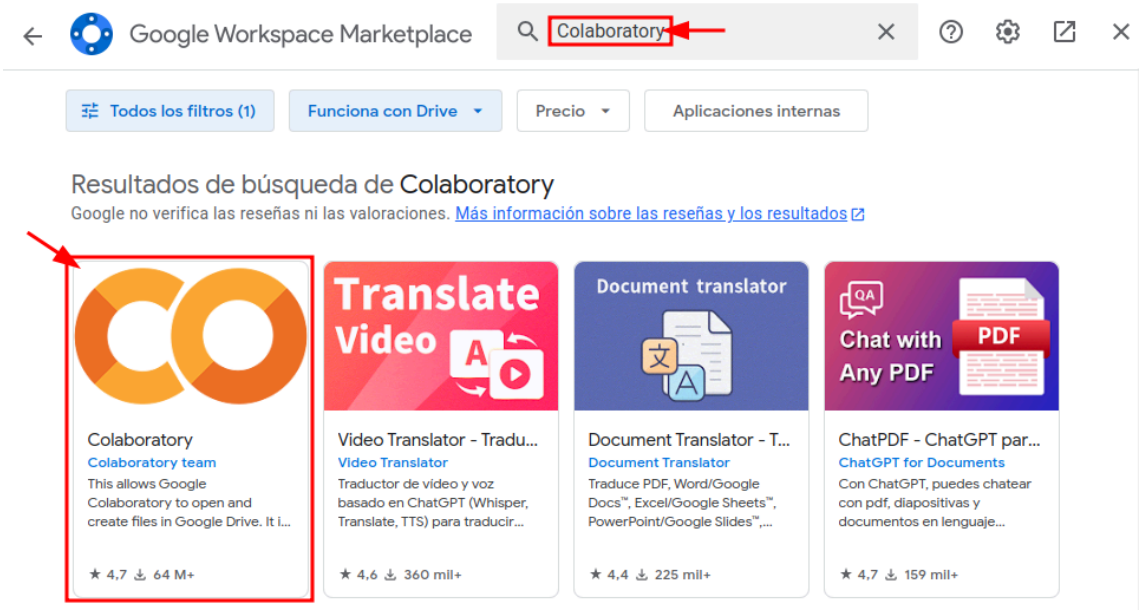
Uso de Google Colab

Para comenzar el módulo lo primero que tenemos que hacer es asegurarnos de que tenemos disponible **Google Collaboration** también conocido simplemente como **Colab**. Para ello basta con acceder a nuestra cuenta de Google Drive (es necesario tener una cuenta de correo de Google Gmail).

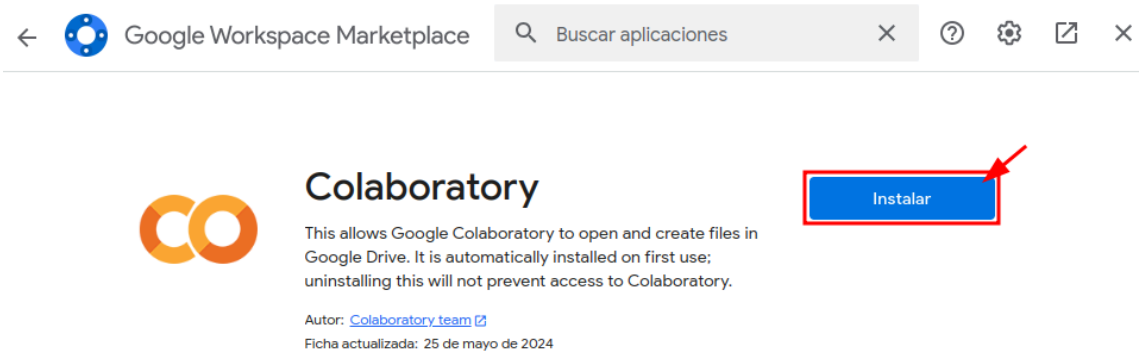
Vamos a cualquiera de nuestras carpetas de "*Mi unidad*" y hacemos click con el botón derecho sobre el área en blanco del centro de la pantalla, seleccionamos la opción "*Más*" y luego la opción "*Conectar más aplicaciones*".



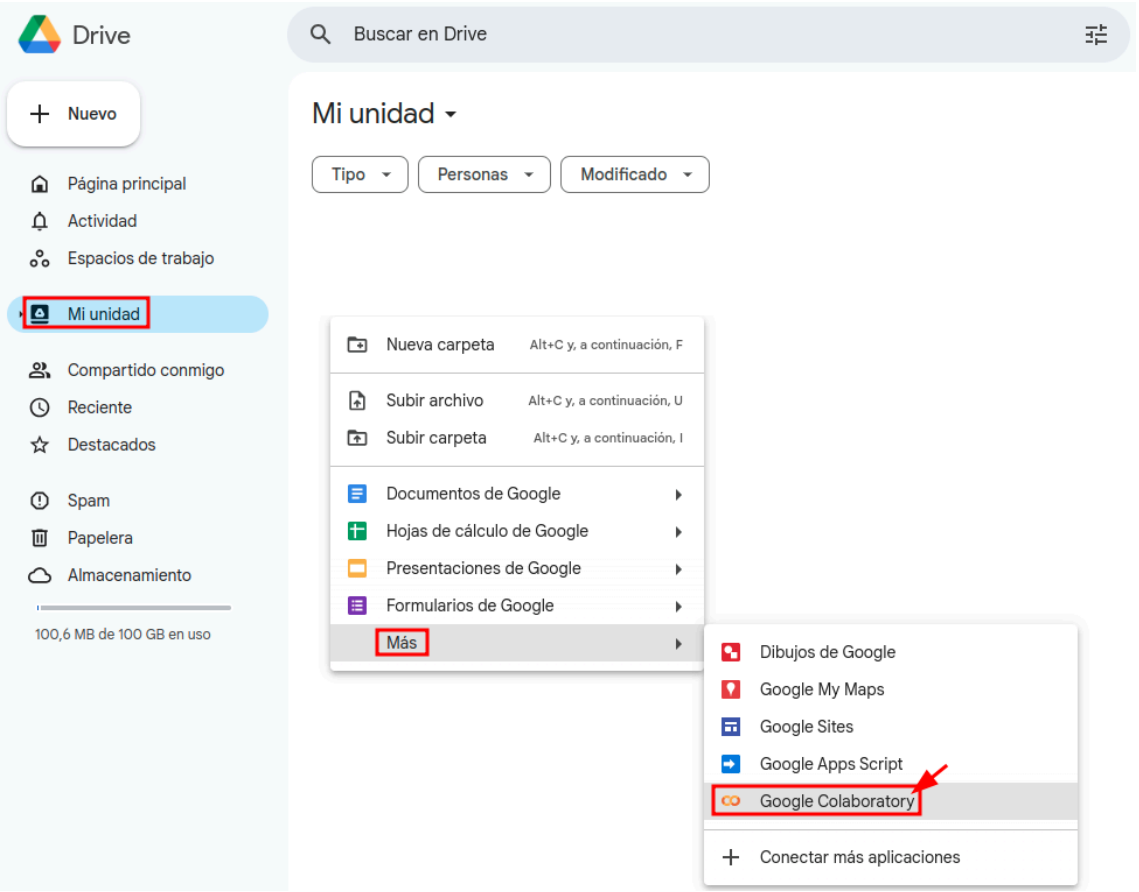
Se nos abrirá una ventana emergente con el "*Google Workspace Marketplace*". En la barra de búsqueda debemos escribir "*Colaboratory*" y tendremos que hacer click sobre el icono de la aplicación "*Colaboratory*" tal y como se muestra en la siguiente imágen.



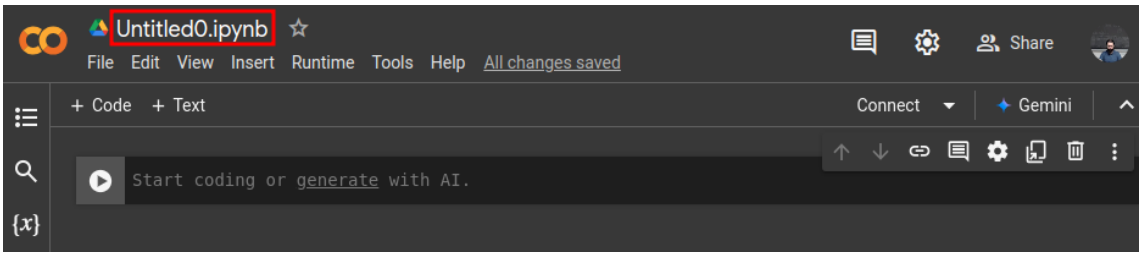
Por último, pulsamos el botón "*Instalar*", de este modo se instalará la aplicación de Google Colab en nuestro Drive y lo tendremos disponible para crear archivos de tipo Google Colab.



Ahora es el momento de crear nuestro primer archivo Colab, para ello haremos botón derecho en cualquier carpeta de nuestro Google Drive, seleccionamos la opción "*Más*" y ahora ya aparece el tipo de archivo a crear "*Google Colaboratory*", tal y como se muestra en la siguiente imágen.



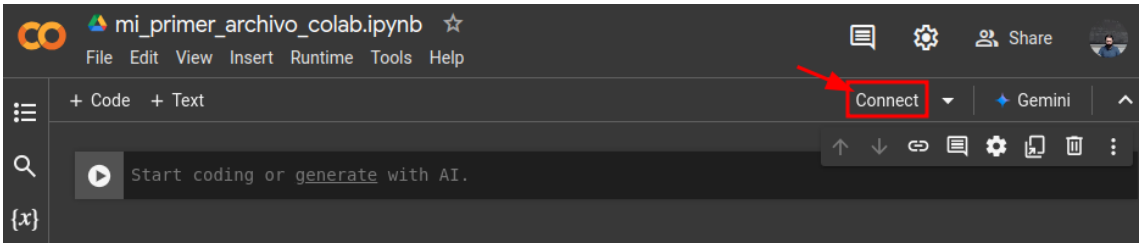
Se nos abrirá un archivo que por defecto tendrá como nombre "*Untitled0.ipynb*", pero podremos cambiarlo simplemente haciendo doble click sobre el nombre del archivo.



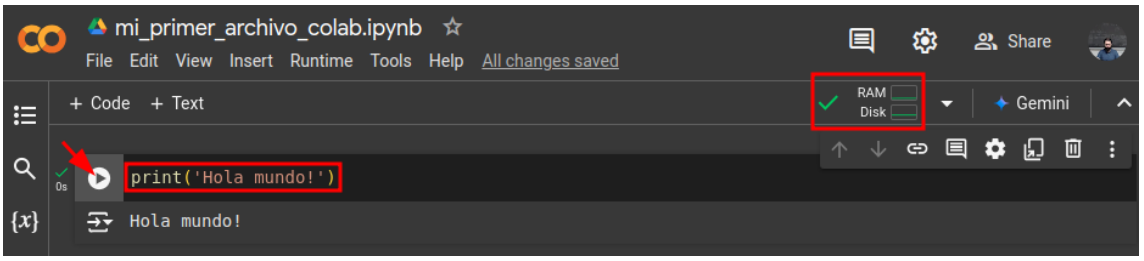
Cambiamos el nombre, por ejemplo "*mi_primer_archivo_colab.ipynb*". Es muy importante no cambiarle la extensión "*.ipynb*", debemos dejarla siempre así.

Ahora deberemos conectar nuestro Colab a un servidor virtual temporal de los que nos ofrece Google, no os preocupéis por entender esta parte, simplemente sabed que se trata de un PC en la nube de Google que ya tiene

instalado Python y está listo para que lo utilicemos. Simplemente pulsamos el botón donde pone “Connect” y tardará unos segundos en conectarse.



Una vez se conecte aparecerá un check verde indicando que se nos ha asignado una máquina virtual con memoria *RAM* y *Disk* temporal para poder trabajar con este Colab. En la cajita donde pone “*Start coding or generate with AI*” ya podremos comenzar a escribir código Python, por ejemplo “*print(‘Hola mundo!’)*”, y finalmente pulsamos el botón “*Run*” o la combinación de teclas “*Ctrl+Intro*” para ejecutar nuestro código, tal y como se muestra en la imágen.



A partir de este momento ya tenemos el entorno de trabajo listo para continuar con los ejemplos de clase.

Métodos de las colecciones

En este punto revisaremos algunos de los métodos más utilizados en los tipos de datos y colecciones que se vieron en el módulo de “*Fundamentos de Python*”. Existen muchos más métodos de los que se tratarán en este punto, pero se tratarán aquellos que son más relevantes o más usados por la mayoría de los usuarios.

El alumno podrá escribir cada código de ejemplo en una de las cajitas de ejecución de su archivo Colab y al ejecutar el código se mostrará el resultado. A continuación se pondrán algunos breves ejemplos de cada tipo de dato o colección seguido de su resultado.

Métodos en cadenas de texto

Todos los caracteres alfabéticos en mayúsculas:

```
'Hola Mundo'.upper()
'HOLA MUNDO'
```

Todos los caracteres alfabéticos a minúsculas:

```
'Hola Mundo'.lower()
'hola mundo'
```

Primera letra del texto en mayúscula:

```
'hola mundo'.capitalize()
'Hola mundo'
```

Primera letra de cada palabra en mayúscula:

```
'hola mundo'.title()
'Hola Mundo'
```

Contabilizar el número de veces que aparece una subcadena o carácter dentro de una cadena:

```
'Hola mundo'.count('o')
2
'Hola mundo'.count('mundo')
1
```

Buscar los índices de aparición de una subcadena, es decir, el lugar en el que aparecen:

```
'Hola mundo'.find('mundo')
5
```

Buscar el índice de la última aparición de una subcadena:

```
'Hola mundo mundo mundo'.rfind('mundo')
17
```

Comprobar si una cadena de texto está compuesta únicamente por números:

```
c = '100'
c.isdigit()
True
```

Comprobar si una cadena de texto está compuesta por carácteres alfanuméricos:

```
c = 'abcd1234'
c.isalnum()
True
```

Comprobar si una cadena de texto está compuesta únicamente por letras:

```
c = 'abcd'
c.isalpha()
True
```

Aquí podemos ver cómo devuelve un valor `False` porque el carácter “espacio” que hay entre la palabra `Hola` y `mundo` no es una letra.

```
c = 'Hola mundo'
c.isalpha()
False
```

Comprobar si todos los carácteres son letras minúsculas:

```
c = 'hola mundo'
c.islower()
True
```

Comprobar si todos los carácteres son letras mayúsculas:

```
c = 'HOLA MUNDO'
c.isupper()
True
```

Comprobar si la primera letra de cada palabra es mayúscula:

```
c = 'Hola Mundo'
c.istitle()
True
```

Comprobar si una cadena está compuesta por espacios o tabulaciones:

```
c = '   '
c.isspace()
True
```

Comprobar si una cadena comienza por un carácter o subcadena concreta:

```
c ='Hola mundo'
c.startswith('H')
```

```
True
c.startswith('Hola')
True
```

Comprobar si una cadena termina con un carácter o subcadena concreta:

```
c.endswith('o')
True
c.endswith('mundo')
True
```

Separar una cadena en una lista de subcadenas a partir un caracter que haga de delimitador, por ejemplo el espacio:

```
c ='Hola mundo mundo'
c.split()
['Hola', 'mundo', 'mundo']
```

El mismo ejemplo pero con el carácter ; como delimitador:

```
c ='aaa;bbb;ccc'
c.split(';')
['aaa', 'bbb', 'ccc']
```

Añadir un carácter o subcadena entre cada carácter de una cadena, por ejemplo una coma o un guión bajo:

```
','.join(c)
'H,o,l,a, ,m,u,n,d,o'
'_'.join(c)
'H_o_l_a_ _m_u_n_d_o'
```

Eliminar todos los carácteres o subcadenas que aparezcan al inicio y al final de una cadena, por ejemplo espacios:

```
c ='     Hola mundo   '
c.strip()
'Hola mundo'
```

Reemplazar un carácter o una subcadena de una cadena, por ejemplo, cambiar la letra `o` por ceros, o la palabra `mundo` por `Javier`:

```
c ='Hola mundo'
c.replace('o', '0')
'H0la mund0'
c.replace('mundo', 'Javier')
'Hola Javier'
```

También podemos eliminar un caracter o una subcadena un número de veces:

```
c ='Hola mundo mundo mundo mundo mundo'
c.replace(' mundo', '', 3)
'Hola mundo mundo'
```

Métodos en listas

Añadir elementos a una lista:

```
l = [1, 2, 3]
l.append(4)
l
[1, 2, 3, 4]
```

Eliminar todos los elementos de una lista:

```
l = [1, 2, 3]
l.clear()
l
[]
```

Unir los elementos de dos listas en una:

```
l1 = [1, 2, 3]
l2 = [4, 5, 6]
l1.extend(l2)
l1
[1, 2, 3, 4, 5, 6]
```

Contar cuántas veces aparece un elemento en una lista:

```
l = ['Hola', 'mundo', 'mundo']
l.count('mundo')
2
```

Mostrar la posición del índice en la que aparece por primera vez un elemento en una lista.

```
l = ['Hola', 'mundo', 'mundo']
l.index('Hola')
0
l.index('mundo')
1
```

Insertar un elemento dentro de una lista en una posición indicada:

```
l = [5, 10, 15, 25]
```

```
l.insert(3, 20)
l
[5, 10, 15, 20, 25]
```

Sacar un elemento en una posición indicada de una lista. Si no se indica ninguna posición sacará el último elemento.

```
l = [10, 20, 30, 40, 50]
l.pop()
50
l [10, 20, 30, 40]
l.pop(1)
20
l
[10, 30, 40]
```

Borrar un elemento de la lista indicando el propio elemento. Si hay varios solo borra el primero:

```
l = ['uno', 'dos', 'tres']
l
['uno', 'dos', 'tres']
l.remove('dos')
l
['uno', 'tres']
```

Invertir el orden de los elementos de una lista:

```
l = [1, 2, 3, 4, 5]
l.reverse()
l
[5, 4, 3, 2, 1]
l = ['uno', 'dos', 'tres']
l.reverse()
l
['tres', 'dos', 'uno']
```

Ordenar elementos de una lista:

```
l = [3, -15, 27, -9, 0]
l.sort()
l
[-15, -9, 0, 3, 27]
l = ['bbb', 'eee', 'ttt', 'ccc']
l.sort()
l
['bbb', 'ccc', 'eee', 'ttt']
```

Métodos en diccionarios

Obtener un valor por defecto cuando queremos acceder a una clave que no existe en un diccionario:

```
colores = {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
colores.get('amarillo', 'No se encuentra')
'yellow'
colores.get('negro', 'No se encuentra')
'No se encuentra'
```

Obtener una lista con las claves de un diccionario:

```
colores = {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
colores.keys()
dict_keys(['amarillo', 'azul', 'verde'])
```

Obtener una lista con los valores de las claves de un diccionario:

```
colores = {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
colores.values()
dict_values(['yellow', 'blue', 'green'])
```

Obtener una lista de tuplas con la clave y valor de cada elemento de un diccionario:

```
colores = {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
colores.items()
dict_items([('amarillo', 'yellow'), ('azul', 'blue'), ('verde', 'green')])
```

Sustraer o eliminar una clave y su valor de un diccionario:

```
colores = {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
colores.pop('amarillo')
'yellow'
colores {'azul': 'blue', 'verde': 'green'}
```

Si quisiéramos extraer de un diccionario un elemento o registro que no existe, por ejemplo `negro`, podríamos añadir al método `pop()` un texto a mostrar en este caso:

```
colores = {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
colores.pop('negro', 'No se encuentra')
'No se encuentra'
```

Vaciar o eliminar todos los elementos de un diccionario:

```
colores = {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
colores {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
colores.clear()
colores
{}
```

Módulos

Los módulos son archivos que contienen definiciones y declaraciones en lenguaje Python. De esta manera es posible importarlos en otros scripts o programas y reutilizar estas funcionalidades.

Collections

El módulo integrado `collections` de una serie de colecciones muy interesantes que podremos utilizar para multitud de acciones bastante recurrentes. Para poder hacer uso de sus colecciones podemos importar la colección en cuestión de la siguiente manera:

```
from collections import Counter
```

Por ejemplo, la colección `Counter` nos devolverá un diccionario con el número de veces que se repite cada uno de los elementos de una lista. Vamos a ver algunos ejemplos:

```
lista = [1, 2, 4, 3, 3, 5, 1, 3, 1, 1, 6]
Counter(lista)
```

```
Counter({1: 4, 3: 3, 2: 1, 4: 1, 5: 1, 6: 1})
```

En este resultado se puede observar que el número `1` aparece cuatro veces, el número `3` tres veces, el número `2` una vez, etc.

Otro ejemplo en el uso de la colección `Counter` es contar las veces que aparece un carácter en una cadena de caracteres o *string*:

```
cadena = 'Hola mundo!'
Counter(cadena)
```

```
Counter({'o': 2, 'H': 1, 'l': 1, 'a': 1, ' ': 1, 'm': 1, 'u': 1, 'n': 1, 'd':
1, '!': 1})
```

En este caso el programa nos dice que el carácter `'o'` aparece dos veces, el carácter `'H'` una vez, el caracter `'l'` una vez, etc.

Podría darse el caso en el que tenemos una cadena de caracteres con una varias palabras separadas por un espacio, por ejemplo:

```
cadena = 'rojo verde azul rojo morado rojo blanco blanco'
```

En este caso queremos saber cuántas veces aparece cada palabra, pero hay que precisar que esto no es una lista de palabras, sino una cadena de carácteres. Para resolverlo primero podemos pasar esta cadena de caracteres compuesta de palabras separadas por espacio a una lista, y para ello podemos usar la función integrada `split()`, al que si no le pasamos ningún argumento tomará el carácter espacio por defecto:

```
cadena.split()
```

```
['rojo', 'verde', 'azul', 'rojo', 'morado', 'rojo', 'blanco', 'blanco']
```

Ahora que ya tenemos cada palabra como un elemento de una lista ya podemos utilizar la colección `Counter` del mismo modo que antes:

```
Counter(cadena.split())
```

```
Counter({'rojo': 3, 'blanco': 2, 'verde': 1, 'azul': 1, 'morado': 1})
```

De este modo obtendremos cuántas veces aparece cada palabra. Podríamos aprovechar este contador de elementos para utilizar otra función integrada llamada `most_common()`, a la que le hemos de pasar como argumento el número de elementos que queremos que nos diga que son los más comunes, por ejemplo `1`:

```
lista = [10, 20, 30, 40, 10, 20, 30, 10, 20, 10]
c = Counter(lista)

c.most_common(1)
```

```
[(10, 4)]
```

Nos dice que el elemento más común es el número `10`, que aparece cuatro veces. Si a la función integrada `most_common()` le pasamos como argumento el número `2` nos devolverá los dos elementos más comunes:

```
c.most_common(2)
```

```
[(10, 4), (20, 3)]
```

Datetime

Uno de los módulos más interesantes sin duda es `datetime`, que nos servirá para manejar y trabajar con información relacionada con las fechas. Para trabajar sobre este punto vamos a comenzar importando este módulo de la siguiente manera:

```
import datetime
```

Ahora vamos a crear un objeto de tipo `datetime` en el que haremos uso del subpaquete `datetime` y su método `now()`:

```
dt = datetime.datetime.now()

print(dt)
```

```
2024-11-05 18:09:49.774602
```

De esta manera la variable `dt` nos devuelve la fecha y hora actual. También podemos acceder a cada uno de los atributos del objeto `dt`, como solo el año, el mes, día, hora, minuto, segundo o microsegundos, de la siguiente manera:

```
print(dt.year)
print(dt.month)
print(dt.day)
print(dt.hour)
print(dt.minute)
print(dt.second)
print(dt.microsecond)
```

```
2024
11
5
18
10
10
514855
```

Ahora que ya sabemos cómo podemos acceder a cada uno de los elementos de los que se compone una fecha y hora podremos darle el formato que se prefiera, por ejemplo:

```
print(f'{dt.year}/{dt.month}/{dt.day}')
print(f'{dt.hour}:{dt.minute}:{dt.second}.{dt.microsecond}')
```

```
2024/11/5
18:10:24.277705
```

También podemos crear una fecha manualmente tal y como se muestra en el siguiente ejemplo:

```
dt = datetime.datetime(2000, 1, 1, 0, 0)

print(dt)
```

```
2000-01-01 00:00:00
```

Si queremos cambiar el año asignando un nuevo valor al atributo `year` podríamos hacerlo con el método `replace()` de la siguiente manera:

```
dt = dt.replace(year=3000)

print(dt)
```

```
3000-01-01 00:00:00
```

En el módulo `datetime` existe un método llamado `isoformat()` que convierte el dato de tipo fecha a un *standard* ISO, por ejemplo:

```
dt = datetime.datetime.now()

print(dt.isoformat())
```

```
2024-11-05T18:11:03.518895
```

Otro método para darle formato personalizado a la fecha y hora es el método `strftime()`, por ejemplo:

```
print(dt.strftime('%A %d %B %Y %I:%M'))
```

```
Tuesday 05 November 2024 06:12
```

`%A` es el día de la semana escrito en inglés, `%d` es el número de día del mes, `%B` es el nombre del mes en inglés, `%Y` es el año con 4 cifras, `%I` es la hora (en formato 12h, para formato 24h es `%H`) y `%M` son los minutos.

Random

El módulo `random` es un módulo que contiene varias herramientas o funcionalidades para trabajar y generar números aleatorios. Se utiliza mucho en el desarrollo de videojuegos o en desarrollos en los que se necesita cierto grado de seguridad. Veamos algunos ejemplos. Lo primero que hay que hacer es importar el módulo `random` de la siguiente manera:

```
import random
```

Ahora que ya tenemos el módulo importado podremos empezar a generar un número aleatorio de forma sencilla con el siguiente ejemplo:

```
print(random.random())
print(random.random())
print(random.random())
```

```
0.06823749155608883
0.9070119606268106
0.4445508707984924
```

De esta forma podremos generar números flotantes aleatorios menores o iguales que uno y mayores de cero. Si quisiéramos números random entre un rango de dos números podríamos usar el método `uniform()` y solo tendríamos que pasar estos dos números como argumentos, por ejemplo números random entre uno y diez:

```
print(random.uniform(1, 10))
print(random.uniform(1, 10))
print(random.uniform(1, 10))
```

```
2.382198826548743
4.8697236381240865
3.8781201434396864
```

Otra manera de generar números enteros random entre cero y un número es utilizando el método `randrange()`, por ejemplo, entre cero y diez:

```
print(random.randrange(10))
print(random.randrange(10))
print(random.randrange(10))
```

```
8
1
4
```

También podemos pasarle dos números como argumentos para que devuelva un número aleatorio entre esos números, por ejemplo:

```
print(random.randrange(0, 100))
print(random.randrange(0, 100))
print(random.randrange(0, 100))
```

```
95
52
91
```

Y si añadimos un número `2` como tercer argumento solo nos sacará números random pares entre cero y cien:

```
print(random.randrange(0, 100, 2))
print(random.randrange(0, 100, 2))
print(random.randrange(0, 100, 2))
```

```
94
46
32
```

Además de poder usar el módulo `random` con números también podremos usarlo con algunas colecciones como las cadenas de texto, en la que podremos escoger una letra de forma aleatoria. Esto se consigue con el método `choice()`, por ejemplo:

```
cadena = 'Hola mundo!'

print(random.choice(cadena))
print(random.choice(cadena))
print(random.choice(cadena))
```

```
n
u
o
```

El método `choice()` también nos vale para listas, en este caso se obtendría de forma aleatoria cualquiera de los elementos de la lista, por ejemplo:

```
lista = [1, 2, 3, 4, 5]

print(random.choice(lista))
print(random.choice(lista))
print(random.choice(lista))
```

```
4
5
3
```

Además del método `choice()` también podremos usar un método llamado `shuffle()` para desordenar los elementos de una lista y que permanezcan guardados de ese modo en la lista de origen, mezcla los elementos de forma aleatoria, por ejemplo:

```
lista = [1, 2, 3, 4, 5]

print(lista)
```

```
random.shuffle(lista)

print(lista)
```

```
[1, 2, 3, 4, 5]
[3, 4, 1, 2, 5]
```

Por último también podremos usar un método llamado `sample()` al que le podremos pasar una lista como argumento, y también un número de elementos que queremos que nos devuelva de forma aleatoria, por ejemplo, si quisiéramos que nos devolviese tres elementos de la lista de forma aleatoria podríamos hacerlo de la siguiente manera:

```
lista = [1, 2, 3, 4, 5]

print(random.sample(lista, 3))
print(random.sample(lista, 3))
print(random.sample(lista, 3))
```

```
[4, 5, 1]
[4, 2, 5]
[3, 4, 2]
```

Existen infinidad de ejemplos y un montón de métodos disponibles más en el módulo `random`, estos solo son los más utilizados o los más conocidos.

Manejo de ficheros

Hasta ahora todo lo que hemos visto son pequeños fragmentos de código que almacenan información como variables, constantes u objetos en tiempo de ejecución, es decir, que solo existen mientras el programa se está ejecutando.

Pero hemos llegado a un punto en el que probablemente nos interese almacenar algunos de los datos con los que hemos aprendido a trabajar en algún fichero, de modo que al cerrar o apagar el programa estos queden persistentemente en un archivo que luego podría volver a cargarse al ejecutar el programa de nuevo, de ese modo no se perdería la información.

Python nos permite realizar las siguientes operaciones con ficheros:

- Creación
- Apertura o lectura
- Modificación
- Cierre

Creación

Comenzamos con un fragmento de código que escriba una línea de texto en un archivo, de modo que si el archivo no existe lo cree. Probemos con el siguiente código:

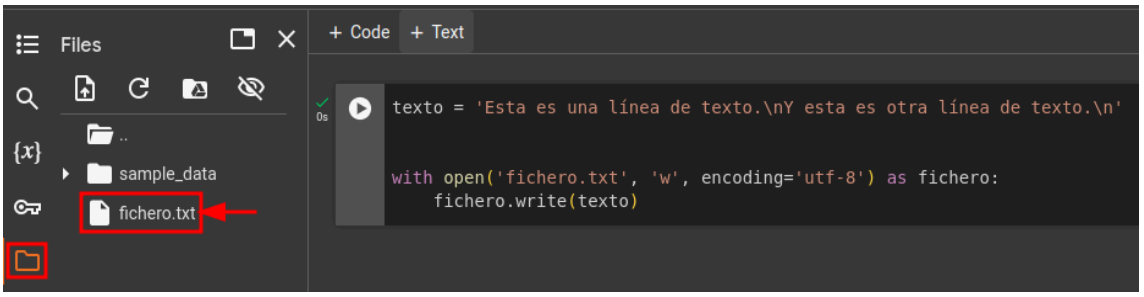
```
texto = 'Esta es una línea de texto.\nY esta es otra línea de texto.\n'

with open('fichero.txt', 'w', encoding='utf-8') as fichero:
    fichero.write(texto)
```

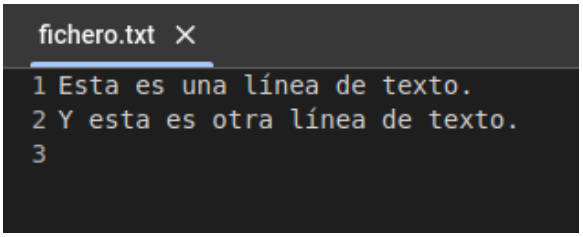
En este ejemplo hemos creado una variable llamada `texto` con las líneas de texto que vamos a escribir en el fichero, luego con la sentencia `with` llamamos a la función `open` a la que le hemos pasado tres argumentos, el primero es el nombre del fichero con el que vamos a trabajar `fichero.txt`, el segundo argumento es la modalidad que vamos a utilizar, que en este caso es `w` de escritura en inglés (*write*) y el tercero es la codificación de caracteres `encoding='utf-8'` para evitar tener problemas con los caracteres que puedan contener acentos. Esta modalidad creará el archivo si no existe y escribirá el texto dentro. Si el archivo existiera lo sobrescribirá.

A continuación se invoca a al método `write()` del objeto `fichero` al que le pasaremos como argumento nuestra variable `texto`, de este modo se escribirá en el archivo que hemos especificado.

Si ejecutamos nuestro código veremos que no hay salida por pantalla alguna, pero si listamos los archivos que se encuentran en el directorio actual de la máquina virtual temporal que nos ha asignado Google, podremos ver que se ha creado un archivo nuevo llamado `fichero.txt`.



Si quisiéramos abrir el archivo para ver su contenido podemos hacer doble click sobre el archivo. De ese modo podremos comprobar que en su interior aparece nuestras líneas de texto de la variable `texto`.



IMPORTANTE: Hay que tener en cuenta que los archivos que se generen de este modo, como `fichero.txt`, se almacenarán de forma temporal mientras tengamos corriendo el Colab. En el momento en el que nos desconectemos o cerremos el Colab estos archivos se podrán perder.

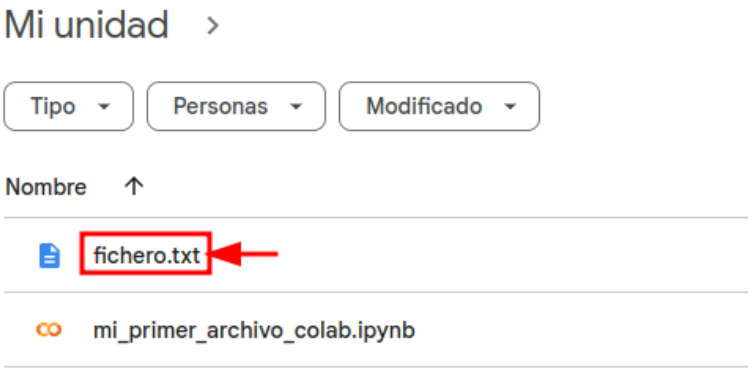
Si quisiéramos que el archivo generado se almacene permanentemente en nuestra carpeta de Google Drive en la que tenemos el archivo Colab debemos ejecutar previamente el siguiente código:

```
from google.colab import drive
import os

drive.mount('/content/drive')

mi_directorio = '/content/drive/My Drive'
os.chdir(mi_directorio)
```

Si ahora volvemos a ejecutar el código que genera el archivo `fichero.txt` veremos se ha generado dicho archivo en nuestra carpeta de Google Drive.



Apertura o lectura

Para abrir un fichero ya existente y leer su contenido se puede hacer de la siguiente manera:

```
with open('fichero.txt', 'r', encoding='utf-8') as fichero:
    texto = fichero.read()

print(texto)
```

Con esta modalidad `'r'` abrimos el fichero en modo lectura (*read*). Almacenamos en una variable llamada `texto` el contenido del fichero, y finalmente, fuera del contexto del `with` imprimimos el contenido de la variable `texto` con el siguiente resultado:

```
Esta es una línea de texto.
Y esta es otra línea de texto.
```

Como se puede ver, estaríamos almacenando en la variable `texto` el contenido del fichero. Podemos hacer la prueba de editar manualmente el contenido del fichero y volver a ejecutar el programa, igualmente se leerán todas las líneas del fichero.

Una manera de leer un archivo línea a línea es almacenando cada línea en una lista. Esto se puede hacer con un método llamado `readlines()` que tienen los objetos de tipo archivo, por ejemplo:

```
with open('fichero.txt', 'r', encoding='utf-8') as fichero:
    lineas = fichero.readlines()

print(lineas)
```

```
['Esta es una línea de texto.\n', 'Y esta es otra línea de texto.\n']
```

Modificación

Además de abrir un fichero y escribir en él un texto o leer, también podemos abrir un fichero existente y añadir nuevas líneas. Esto se consigue mediante una modalidad `'a'` que añade líneas al final (*append*), por ejemplo:

```
with open('fichero.txt', 'a', encoding='utf-8') as fichero:
    fichero.write('Esta es una línea nueva.\n')
```

Esta modalidad no solo sirve para añadir líneas al final del archivo, si no que también lo crea si este no existe.

Por último, podemos leer un fichero y luego procesar cada línea de forma independiente de la siguiente manera:

```
with open('fichero.txt', 'r', encoding='utf-8') as fichero:
     for linea in fichero:
         print(linea.rstrip('\n'))
```

Salida:

```
Esta es una línea de texto.
Y esta es otra línea de texto.
Esta es una línea nueva.
```

El método `.rstrip('\n')` elimina en cada línea el último caracter que sea un salto de línea.

Pandas

Pandas es una librería de Python que se utiliza para el análisis y manipulación de datos. La librería Pandas ofrece una serie de herramientas y estructuras de datos que permiten a los usuarios trabajar con datos de una manera más eficiente y rápida. Una de las principales ventajas de Pandas es que permite trabajar con datos de diferentes tipos, como tablas numéricas o series de tiempo, y ofrece una serie de herramientas para manipular y analizar esos datos de manera eficiente. Tiene como base dos estructuras de datos, las `Series` y los `DataFrames`.

Importación de Pandas

Por convención la importación de la librería Pandas en nuestro código se realiza de la siguiente manera:

```
import pandas as pd
```

A partir de este momento podremos utilizar el alias `pd` para invocar cualquier método, atributo o herramienta incluída en la librería Pandas.

En los siguientes puntos veremos en detalle las `Series` y los `DataFrames` con algunos ejemplos, pero antes veamos cómo usar la librería **Matplotlib**.

Importación y visualización con Matplotlib

Matplotlib es una librería de visualización que tiene una gran variedad de gráficos y que es fácilmente configurable. Más información acerca de los gráficos disponibles en https://matplotlib.org/gallery/index.html. Para utilizar esta librería debemos importarla de la siguiente manera:

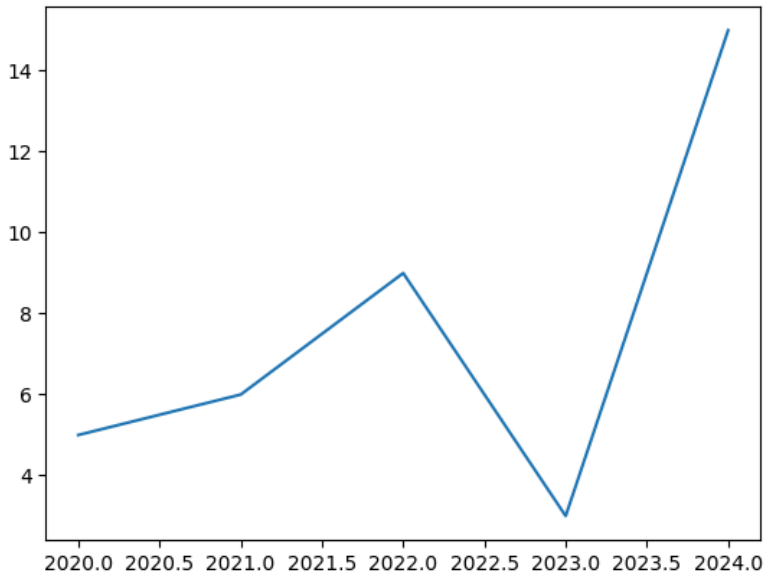
```
import matplotlib.pyplot as plt
```

A partir de este momento podremos utilizar en nuestro código el alias `plt` para acceder a todos los métodos de esta librería. Veamos un ejemplo en el que cargaremos un array de datos para el eje `x`, llamado por ejemplo `year`, y otro array de datos para el eje `y`, llamado por ejemplo `value`:

```
year = [2020, 2021, 2022, 2023, 2024]
value = [5, 6, 9, 3, 15]
```

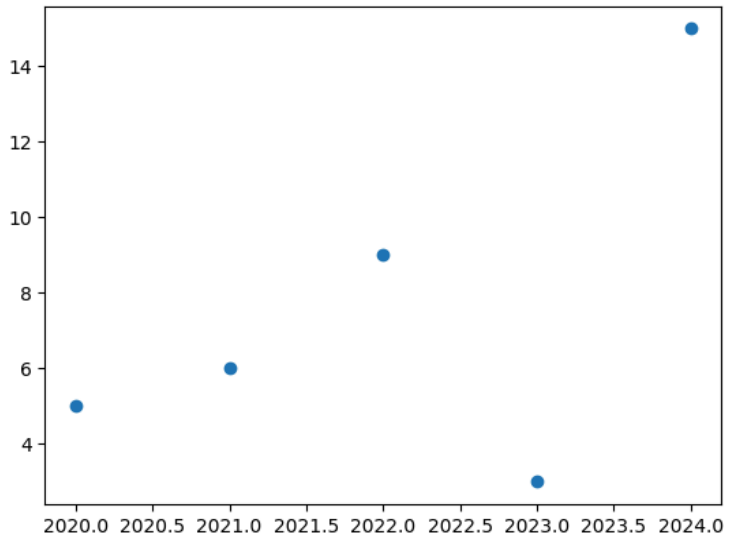
Finalmente podremos generar un gráfico de líneas mediante el método `plot()` de la librería `matplotlib`, al que le tendremos que pasar como argumentos primero el array de datos que queremos utilizar en el eje `x` y segundo el array que usaremos para el eje `y`:

```
plt.plot(year, value)
```



Si quisiéramos generar otro tipo de gráfico con los mismos datos podríamos utilizar por ejemplo el método `scatter()` al que también hay que pasarle como argumentos los datos de los ejes `x` e `y`:

```
plt.scatter(year, value)
```



Esta sería una manera de crear gráficos muy sencillos a partir de un par de listas con datos, pero normalmente se suelen utilizar fuentes de datos más grandes y complejas como son los dataframes de Pandas que hemos visto anteriormente. Para visualizar con la librería `matplotlib` la información de un dataframe de Pandas podemos hacerlo de la siguiente manera. Primero importamos el dataframe, en este caso uno llamado `hum_temp.csv` con algunos datos random relativos a humedad y temperatura:

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv(
    r'datasets/hum_temp.csv',
    encoding='ISO-8859-1',
    delimiter=';'
)
```

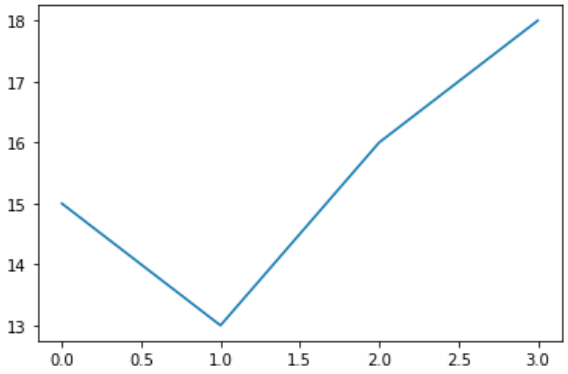
Comprobamos que se ha cargado correctamente:

```
df
```

	humidity	temperature
0	60	15
1	80	13
2	45	16
3	55	18

A continuación usamos el método `plot()` de la instancia `plt` al que le pasaremos el nombre del dataframe, en este caso `df` y entre corchetes el nombre de la columna que queremos utilizar para representarlo en el eje `y`, si no se especifica el eje `x` en este se utilizarán los valores del índice de los registros de datos:

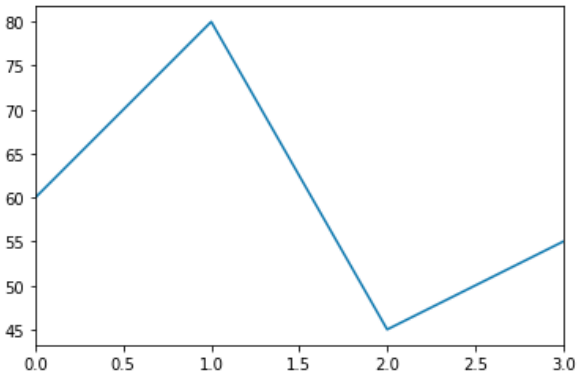
```
plt.plot(df['temperature'])
```



Como se puede ver, los datos de la columna `temperature` aparecen en el eje `y` y el índice de los datos en el eje `x`.

Veamos una manera mejor de representar los datos de este dataframe, en este caso usaremos el método `plot()` con el dataframe `df` de la siguiente manera:

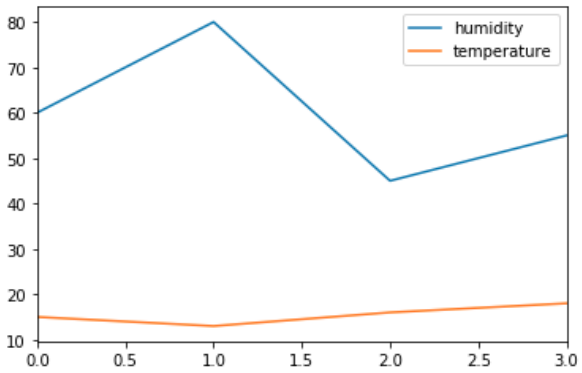
```
df['humidity'].plot()
```



Si eliminamos el nombre de la columna entre corchetes y especificamos únicamente el nombre del dataframe `df` podremos ver en el mismo gráfico

todos los registros de cada columna en líneas diferentes, cada una con un color distinto.

```
df.plot()
```



Esta es una manera muy sencilla de generar un gráfico individual que representa un conjunto de datos sencillos, pero ¿y si quisiéramos representar varios gráficos juntos? Matplotlib dispone de una función `subplot()` que permite agrupar varios gráficos en una malla o matriz. Por ejemplo, para generar una maya de 4 gráficos diferentes debemos crear un conjunto de gráficos en una sola figura de la siguiente manera.

1. Creamos una figura de tipo `sublopt` con una malla de 2 filas por 2 columnas.

```
fig, axs = plt.subplots(2, 2)
```

2. Colocamos el primer gráfico en la esquina superior izquierda mediante las coordenadas `0, 0` y añadimos al plot una serie de datos cualquiera, por ejemplo `[1, 2, 3]` y `[2, 4, 3]`. Además le añadimos un título al gráfico para luego poder identificarlo fácilmente en la malla.

```
axs[0, 0].plot([1, 2, 3], [2, 4, 3])
axs[0, 0].set_title('Gráfico 1')
```

3. De la misma manera creamos el segundo gráfico en la esquina superior derecha, indicando las coordenadas `0, 1`. Para representar los datos con gráficos de diferentes tipos, en esta ocasión en vez de un gráfico de

tipo plot usaremos uno de tipo scatter, para que represente los datos en forma de puntos.

```
axs[0, 1].scatter([1, 2, 3], [2, 4, 3])
axs[0, 1].set_title('Gráfico 2')
```

4. El tercer gráfico será de tipo `bar` y representará los datos `['A', 'B', 'C']` y `[3, 2, 5]` en forma de barras. Lo colocaremos en la esquina inferior izquierda con las coordenadas `1, 0`.

```
axs[1, 0].bar(['A', 'B', 'C'], [3, 2, 5])
axs[1, 0].set_title('Gráfico 3')
```

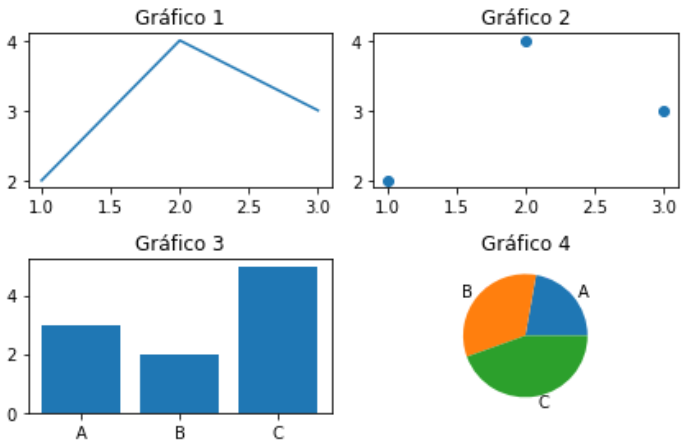
5. Y por último el cuarto gráfico de tipo `pie` en la esquina inferior derecha con las coordenadas `1, 1`.

```
axs[1, 1].pie([2, 3, 4], labels=['A', 'B', 'C'])
axs[1, 1].set_title('Gráfico 4')
```

6. Ajustamos los márgenes y espacios entre los gráficos.

```
fig.tight_layout()
```

Como resultado obtenemos la siguiente figura compuesta de los 4 gráficos que hemos generado anteriormente:



Hasta aquí hemos visto algunas maneras muy sencillas de generar gráficos combinando `pandas` y `matplotlib`. A continuación veremos algunos tipos de datos que ofrece la librería Pandas y que nos vendrán muy bien para trabajar con conjuntos de datos más complejos.

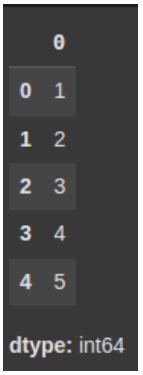
Series

Las `Series` de Pandas son un tipo de datos de la librería Pandas que se utiliza para trabajar con una sola columna de datos. Una serie de Pandas es similar a una lista o un array en otras librerías de Python, pero tiene algunas características adicionales que la hacen más útil para ciertos tipos de análisis de datos.

Por ejemplo, podemos crear una serie de Pandas a partir de una lista de números:

```
serie = pd.Series([1, 2, 3, 4, 5])
serie
```

Esto imprimiría la serie por pantalla de la siguiente forma:



Como se puede ver, la serie contiene una lista de números, junto con un índice que empieza en 0 y aumenta en 1 para cada elemento en la serie. También se puede ver que el tipo de datos de la serie es "int64", lo que significa que se trata de una serie de números enteros de 64 bits.

Dataframes

Un DataFrame es una estructura de datos en Pandas que permite almacenar y manipular datos en forma de tabla con filas y columnas. Es similar a una hoja de cálculo o una tabla en una base de datos relacional. Puede contener distintos tipos de datos, como números enteros o flotantes, cadenas de texto,

fechas y horas, y otros tipos de datos. Cada columna de un DataFrame puede tener un tipo de datos diferente, pero todas las filas en una columna tienen que tener el mismo tipo de datos.

Para crear un DataFrame en Pandas, podemos utilizar la función `pandas.DataFrame()`. Esta función toma como argumentos un diccionario o una lista de diccionarios, cada uno con los datos de una fila en el DataFrame.

Por ejemplo, podríamos crear un DataFrame que contenga información sobre varios libros, como el título, el autor y el número de páginas, de la siguiente manera:

```
df_libros = pd.DataFrame([
    {'título': 'El principito', 'autor': 'Antoine de Saint-Exupéry', 'páginas': 91},
    {'título': 'Don Quijote de la Mancha', 'autor': 'Miguel de Cervantes', 'páginas': 992},
    {'título': 'Cien años de soledad', 'autor': 'Gabriel García Márquez', 'páginas': 417},
    {'título': 'Matar a un ruiseñor', 'autor': 'Harper Lee', 'páginas': 281},
    {'título': 'El gran Gatsby', 'autor': 'F. Scott Fitzgerald', 'páginas': 180},
])

df_libros
```

El resultado sería un DataFrame que contiene las columnas "título", "autor" y "páginas", con una fila para cada libro:

	título	autor	páginas
0	El principito	Antoine de Saint-Exupéry	91
1	Don Quijote de la Mancha	Miguel de Cervantes	992
2	Cien años de soledad	Gabriel García Márquez	417
3	Matar a un ruiseñor	Harper Lee	281
4	El gran Gatsby	F. Scott Fitzgerald	180

En este ejemplo, cada libro se convierte en una fila en el DataFrame, y cada columna contiene los datos correspondientes a ese libro.

Podemos añadir una nueva columna a nuestro dataframe, por ejemplo “ISBN” con un valor por defecto “Unknow” de la siguiente manera:

```
df_libros['ISBN'] = 'Unknown'
```

Y el nuevo DataFrame lucirá de la siguiente manera:

	título	autor	páginas	ISBN
0	El principito	Antoine de Saint-Exupéry	91	Unknown
1	Don Quijote de la Mancha	Miguel de Cervantes	992	Unknown
2	Cien años de soledad	Gabriel García Márquez	417	Unknown
3	Matar a un ruiseñor	Harper Lee	281	Unknown
4	El gran Gatsby	F. Scott Fitzgerald	180	Unknown

Actualmente, el índice de cada fila es un número entero `0`, `1`, `2`, etc, pero podríamos usar cualquier columna del DataFrame para configurarla como índice usando el atributo `set_index()`, como se muestra a continuación:

```
df_libros = df_libros.set_index(['título'])

# Mostramos el contenido del DataFrame.
df_libros
```

título	autor	páginas	ISBN
El principito	Antoine de Saint-Exupéry	91	Unknown
Don Quijote de la Mancha	Miguel de Cervantes	992	Unknown
Cien años de soledad	Gabriel García Márquez	417	Unknown
Matar a un ruiseñor	Harper Lee	281	Unknown
El gran Gatsby	F. Scott Fitzgerald	180	Unknown

Se puede acceder a los datos de dos maneras, es decir, usando el índice de una fila o de una columna:

```
# Usando el índice de una columna.
df_libros['páginas']
```

título	páginas
El principito	91
Don Quijote de la Mancha	992
Cien años de soledad	417
Matar a un ruiseñor	281
El gran Gatsby	180

dtype: int64

```
# Usando el índice de una fila.
df_libros.loc['Cien años de soledad']
```

Cien años de soledad	
páginas	417
ISBN	Unknown

dtype: object

```
# Accediendo a todas las filas de una columna.
df_libros.loc[:, 'autor']
```

	autor
0	Antoine de Saint-Exupéry
1	Miguel de Cervantes
2	Gabriel García Márquez
3	Harper Lee
4	F. Scott Fitzgerald

dtype: object

```
# Accediendo a un elemento específico del DataFrame.
df_libros.loc['Matar a un ruiseñor', 'páginas']
```

```
281
```

También podremos eliminar columnas mediante el comando `del` o el método `drop()`, al que le tendremos que pasar como parámetro `axis` un valor `1` para indicar que queremos eliminar columnas.

```
del df_libros['autor']
```

título	páginas	ISBN
El principito	91	Unknown
Don Quijote de la Mancha	992	Unknown
Cien años de soledad	417	Unknown
Matar a un ruiseñor	281	Unknown
El gran Gatsby	180	Unknown

```
df_libros.drop('ISBN', axis=1)
```

título	páginas
El principito	91
Don Quijote de la Mancha	992
Cien años de soledad	417
Matar a un ruiseñor	281
El gran Gatsby	180

Lectura de ficheros

En esta sección, vamos a utilizar dos archivos CSV con datos: `titles.csv` y `cast.csv`. El archivo `titles.csv` contiene la lista de películas con el año de lanzamiento, y el archivo `cast.csv` tiene cinco columnas que almacenan el título de la película, el año de lanzamiento, el reparto, el tipo (actor/actriz), los personajes y las calificaciones de los actores, como se muestra a continuación.

```
casts = pd.read_csv(
    r'datasets/cast.csv',
    encoding='ISO-8859-1'
)
casts.head()
```

	title	year	name	type	character	n
0	Closet Monster	2015	Buffy #1	actor	Buffy 4	31.0
1	Suuri illusioni	1985	Homo $	actor	Guests	22.0
2	Battle of the Sexes	2017	$hutter	actor	Bobby Riggs Fan	10.0
3	Secret in Their Eyes	2015	$hutter	actor	2002 Dodger Fan	NaN
4	Steve Jobs	2015	$hutter	actor	1988 Opera House Patron	NaN

```
titles = pd.read_csv(
    r'datasets/titles.csv',
    encoding='ISO-8859-1'
)
titles.tail()
```

	title	year
49995	Rebel	1970
49996	Suzanne	1996
49997	Bomba	2013
49998	Aao Jao Ghar Tumhara	1984
49999	Mrs. Munck	1995

- **read_csv**: Lee los datos del archivo CSV.
- **'datasets/<archivo.csv>'**: Ruta donde se encuentra el archivo CSV.
- **encoding='ISO-8859-1'**: Codificación de carácteres ISO-8859-1.
- **head()**: Muestra solo los primeros cinco elementos del DataFrame.
- **tail()**: Muestra solo los últimos cinco elementos del DataFrame.

Si simplemente escribimos el nombre del DataFrame `titles` y pulsamos intro, mostrará las primeras y últimas cinco filas del archivo. Además, al final de la tabla se mostrará el número total de filas y columnas.

```
titles
```

	title	year
0	The Rising Son	1990
1	The Thousand Plane Raid	1969
2	Crucea de piatra	1993
3	Country	2000
4	Gaiking II	2011
...	...	...
49995	Rebel	1970
49996	Suzanne	1996
49997	Bomba	2013
49998	Aao Jao Ghar Tumhara	1984
49999	Mrs. Munck	1995

50000 rows × 2 columns

También podremos utilizar la función `len()` para ver el número total de filas de nuestro DataFrame:

```
len(titles)
50000
```

Operaciones con los datos

Selección

Cualquier fila o columna del DataFrame se puede seleccionar pasando el nombre de la columna o el índice de la fila. Después de seleccionar una columna o fila del DataFrame, se vuelve unidimensional, por lo que se considera una serie.

```
una_serie = titles['title']
type(una_serie)
```

```
pandas.core.series.Series
```

```
una_serie
```

	title
0	The Rising Son
1	The Thousand Plane Raid
2	Crucea de piatra
3	Country
4	Gaiking II
...	...
49995	Rebel
49996	Suzanne
49997	Bomba
49998	Aao Jao Ghar Tumhara
49999	Mrs. Munck

50000 rows × 1 columns

```
otra_serie = titles.loc[0]
type(otra_serie)
```

```
pandas.core.series.Series
```

```
otra_serie
```

	0
title	The Rising Son
year	1990

dtype: object

Filtrado

Los datos se pueden filtrar proporcionando alguna expresión booleana en el DataFrame. Por ejemplo, en el siguiente código, las películas que se lanzaron después de 1985 se filtran del DataFrame `titles` y se almacenan en un nuevo DataFrame al que llamaremos `despues_85`.

```
despues_85 = titles[titles['year'] > 1985]
despues_85
```

	title	year
0	The Rising Son	1990
2	Crucea de piatra	1993
3	Country	2000
4	Gaiking II	2011
5	Medusa (IV)	2015
...	...	...
49990	Junebug	2005
49993	Corruption.Gov	2010
49996	Suzanne	1996
49997	Bomba	2013
49999	Mrs. Munck	1995

29814 rows × 2 columns

Cuando pasamos los resultados booleanos al DataFrame, Pandas mostrará todos los resultados que sean True (*en lugar de mostrar True y False*), tal y como se muestra en el ejemplo anterior. Además, podemos usar "`&`" (and) "`|`" (or) para unir dos condiciones, como se muestra a continuación.

```
peliculas_90 = titles[(titles['year']>=1990) & (titles['year']<2000)]
peliculas_90
```

	title	year
0	The Rising Son	1990
2	Crucea de piatra	1993
12	Poka Makorer Ghar Bosoti	1996
19	Maa Durga Shakti	1999
24	Conflict of Interest	1993
...	...	...
49969	Chi mei wang liang	1998
49979	Gagay: Prinsesa ng brownout	1993
49987	I Won't Dance	1992
49996	Suzanne	1996
49999	Mrs. Munck	1995

4803 rows × 2 columns

Ordenación

Podemos ordenar los datos utilizando los métodos `sort_index()` o `sort_values()`. Veámoslo con un ejemplo. Primero filtramos todas las películas cuyo título sea "`Macbeth`":

```
solo_macbeth = titles[titles['title'] == 'Macbeth']
solo_macbeth
```

	title	year
4226	Macbeth	1913
9322	Macbeth	2006
11722	Macbeth	2013
17166	Macbeth	1997
25847	Macbeth	1998

Si quisiéramos ordenar los datos por los valores de la columna `year` podremos utilizar el método `sort_values()` de la siguiente forma:

```
solo_macbeth = titles[titles['title'] == 'Macbeth'].sort_values('year')
solo_macbeth
```

	title	year
4226	Macbeth	1913
17166	Macbeth	1997
25847	Macbeth	1998
9322	Macbeth	2006
11722	Macbeth	2013

Tratamiento de valores nulos

Es bastante común encontrar algunas columnas que pueden no contener valores. Generalmente se suelen completar con valores `NaN`. Por ejemplo, las filas 3 y 4 del DataFrame `casts` tienen valor `NaN` tal y como se muestra a continuación.

```
casts.loc[3:4]
```

	title	year	name	type	character	n
3	Secret in Their Eyes	2015	$hutter	actor	2002 Dodger Fan	NaN
4	Steve Jobs	2015	$hutter	actor	1988 Opera House Patron	NaN

Estos valores nulos se pueden seleccionar o deseleccionar fácilmente o los contenidos se pueden reemplazar por cualquier otro valor, por ejemplo con cadenas vacías o `0`, etc. En esta sección se muestran varios ejemplos de valores nulos.

El método `isnull()` devuelve `True` si alguna fila tiene valores nulos. Dado que las filas 3 y 4 tienen un valor de `NaN`, por lo tanto, se muestran como `True`.

```
casts['n'].isnull()
```

	n
0	False
1	False
2	False
3	True
4	True
...	...
74996	False
74997	True
74998	False
74999	True
75000	False

75001 rows × 1 columns

Para mostrar las filas completas que contengan algún valor nulo, debemos pasar la condición en el DataFrame.

```
casts[casts['n'].isnull()]
```

	title	year	name	type	character	n
3	Secret in Their Eyes	2015	$hutter	actor	2002 Dodger Fan	NaN
4	Steve Jobs	2015	$hutter	actor	1988 Opera House Patron	NaN
5	Straight Outta Compton	2015	$hutter	actor	Club Patron	NaN
6	Straight Outta Compton	2015	$hutter	actor	Dopeman	NaN
7	For Thy Love 2	2009	Bee Moe $lim	actor	Thug 1	NaN
...	...	...	...	...	...	...
74984	Ovum	2015	Christina Rosas Anastasiou	actress	Fertility Clinic Receptionist	NaN
74985	7 avlu	2009	Evmorfia Anastasiou	actress	Eleni	NaN
74993	I teliki apopliromi	2013	Mara Anastasopoulou	actress	Marina's friend	NaN
74997	The Magician King	2004	Tiannah Anastassiades	actress	Unicycle Race Attendant	NaN
74999	Toxic Tutu	2016	Zoe Anastassiou	actress	Demon of Toxicity	NaN

28966 rows × 6 columns

Los valores `NaN` se pueden completar usando métodos como `fillna()`. En el siguiente ejemplo vamos a reemplazar los valores `NaN` por `N/A`.

```
casts_fill = casts[casts['n'].isnull()].fillna('N/A')
casts_fill
```

	title	year	name	type	character	n
3	Secret in Their Eyes	2015	$hutter	actor	2002 Dodger Fan	N/A
4	Steve Jobs	2015	$hutter	actor	1988 Opera House Patron	N/A
5	Straight Outta Compton	2015	$hutter	actor	Club Patron	N/A
6	Straight Outta Compton	2015	$hutter	actor	Dopeman	N/A
7	For Thy Love 2	2009	Bee Moe $lim	actor	Thug 1	N/A
...	...	...	...	...	...	...
74984	Ovum	2015	Christina Rosas Anastasiou	actress	Fertility Clinic Receptionist	N/A
74985	7 avlu	2009	Evmorfia Anastasiou	actress	Eleni	N/A
74993	I teliki apopliromi	2013	Mara Anastasopoulou	actress	Marina's friend	N/A
74997	The Magician King	2004	Tiannah Anastassiades	actress	Unicycle Race Attendant	N/A
74999	Toxic Tutu	2016	Zoe Anastassiou	actress	Demon of Toxicity	N/A

28966 rows × 6 columns

Operaciones con strings

Podemos realizar varias operaciones con las cadenas de caracteres usando la opción `'.str.'`.

Primero vamos a buscar la película cuyo título es exactamente '`Out`':

```
titles[titles['title'] == 'Out']
```

	title	year
24747	Out	1982
37089	Out	2016

Solo hay dos películas con ese título exacto en nuestro DataFrame. Ahora, busquemos todas las películas que comienzan con '`Out`'. Para ello usaremos la opción '`.str.`' tal y como se muestra a continuación.

```
titles[titles['title'].str.startswith('Out ')]
```

	title	year
2165	Out of Sync	2002
3147	Out of Focus	2010
4809	Out of Depth	2000
5209	Out of Innocence	2016
8317	Out West with the Hardys	1938
10070	Out of Rosenheim	1987
10380	Out of Inferno	2013
11337	Out of the Wreck	1917
12881	Out of Mind	2006
13205	Out to Kill	2014
13268	Out to Sea	1997
18124	Out of Time	2003
20098	Out on a Limb	1992
20333	Out of Balance	2007
20601	Out of Touch	2011
22033	Out of Luck	1923
23445	Out of Reach	2012
24888	Out of Control	1992
27187	Out of the Silent North	1922
27429	Out of Range	2016

Como se puede observar, hay muchísimas más.

Contar valores

Podemos contar el número de ocurrencias usando el método `value_counts()`. Por ejemplo, en el siguiente código mostraremos el número total de películas que hay por cada año.

```
titles['year'].value_counts()
```

year	count
2016	2363
2017	2138
2015	1849
2014	1701
2013	1609
...	...
1900	2
2024	1
1906	1
1905	1
2023	1

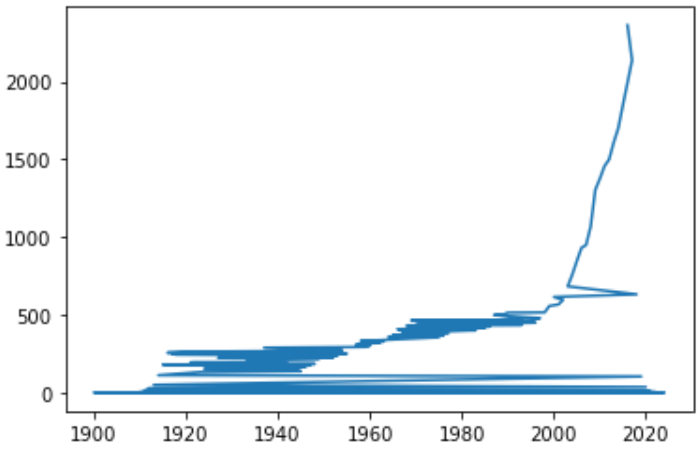
119 rows × 1 columns

dtype: int64

Generación de gráficos

Pandas es compatible con la biblioteca `matplotlib` y también se puede usar para hacer representaciones gráficas de los datos. En la sección anterior filtramos en el DataFrame `titles` el número total de películas por año. En el siguiente código, vamos a guardar estos valores en un nuevo DataFrame al que llamaremos `plot_titles` y luego trazaremos los datos en un gráfico.

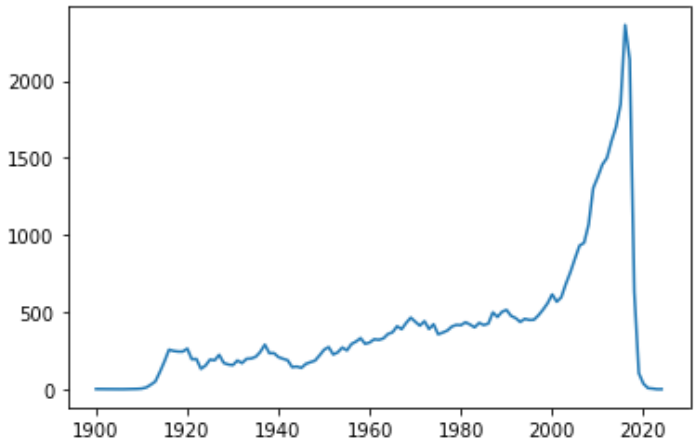
```
plot_titles = titles['year'].value_counts()
plot_titles.plot()
```



Aunque hemos generado un gráfico con los datos correspondientes al número total de películas por año, este no proporciona información demasiado útil.

Es mejor ordenar primero los años (es decir, el índice) y luego trazar los datos como se muestra a continuación.

```
plot_titles.sort_index().plot()
```



El gráfico muestra que la cantidad de películas aumenta cada año, y lo podemos visualizar de una manera más clara respecto al primer gráfico.

Agrupación de datos

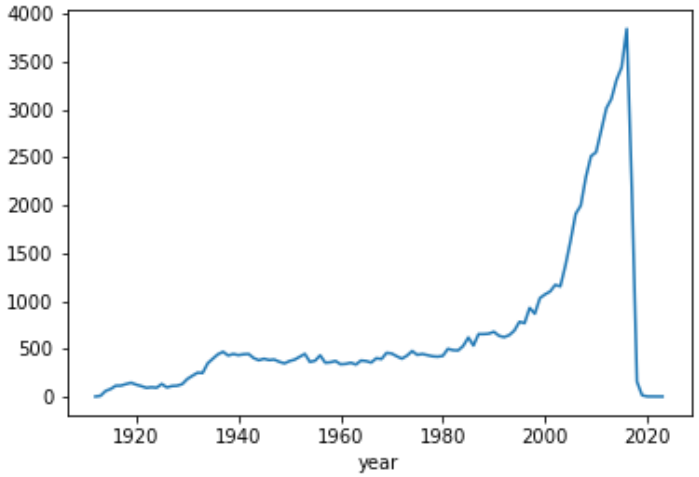
Los datos se pueden agrupar por columnas-cabeceras. Además, se pueden definir formatos personalizados para agrupar los diversos elementos del DataFrame.

Agrupación por nombre de columna

En la sección "*Contar Valores*" anterior, calculamos el valor de las películas/año utilizando el método `count_values()`. También podemos hacerlo con el método `groupby()`.

El método `groupby()` devuelve un objeto, al que luego le concatenamos el método `size()` para contar el número total de filas en cada año. Como se puede ver, el resultado del siguiente código es el mismo que el comando `count_values()` que vimos anteriormente.

```
casts_agrupado = casts.groupby(['year']).size()
casts_agrupado.plot()
```



Además, el método `groupby()` puede tomar varios parámetros para agrupar. Por ejemplo, si queremos agrupar las películas del actor '`Aaron Abrams`' según el año podremos hacerlo de la siguiente forma.

```
casts_aaron_abrams = casts[casts['name'] == 'Aaron Abrams']
casts_aaron_abrams.groupby(['year']).size()
```

year	0
2003	2
2004	2
2005	2
2006	1
2007	2
2008	1
2009	2
2011	5
2013	1
2015	2
2017	2
2018	1

La lista anterior muestra que el año `2003` se encuentra en dos filas con el valor '`Aaron Abrams`' en la columna `name`. Dicho de otra forma, hizo 2 películas en 2003, 2 en 2004, 1 en 2006, etc.

Si queremos ver la lista de películas con los títulos agrupadas por años debemos pasar las columnas `year` y `title` en forma de lista, tal y como se muestra a continuación.

```
casts_aaron_abrams.groupby(['year', 'title']).size()
```

year	title	0
2003	The In-Laws	1
	The Visual Bible: The Gospel of John	1
2004	Resident Evil: Apocalypse	1
	Siblings	1
2005	Cinderella Man	1
	Sabah	1
2006	Zoom	1
2007	Firehouse Dog	1
	Young People Fucking	1
2008	Flash of Genius	1
2009	Amelia	1
	At Home by Myself... with You	1
2011	388 Arletta Avenue	1
	Jesus Henry Christ	2
	Take This Waltz	1
	The Chicago 8	1
2013	It Was You Charlie	1
2015	Closet Monster	1
	Regression	1
2017	#FromJennifer	1
	The Go-Getters	1
2018	Code 8	1

En el código anterior, el método `groupby()` recibe primero en la columna `year` y luego la columna `title`. Primero agrupa todas las películas por año y después, el resultado de este grupo se vuelve a agrupar en función de los títulos. Si nos fijamos bien, en el primer grupo se organizan los datos de los años en orden ascendente, es decir, 2003, 2004 y 2005, etc. Luego en el siguiente grupo se ordenan los datos de los títulos en orden alfabético.

A continuación, agrupamos en función de las calificaciones máximas en un año; es decir, queremos agrupar los artículos por año y ver la calificación máxima en esos años.

```
casts_aaron_abrams.groupby(['year']).n.max()
```

year	n
2003	17.0
2004	9.0
2005	8.0
2006	NaN
2007	1.0
2008	44.0
2009	2.0
2011	4.0
2013	3.0
2015	1.0
2017	1.0
2018	NaN

Los resultados anteriores muestran que la calificación máxima en el año `2003` fue `17.0` para el actor `Aaron Abrams`.

De forma similar podemos calcular las calificaciones mínimas.

```
casts_aaron_abrams.groupby(['year']).n.min()
```

year	n
2003	17.0
2004	9.0
2005	8.0
2006	NaN
2007	1.0
2008	44.0
2009	2.0
2011	4.0
2013	3.0
2015	1.0
2017	1.0
2018	NaN

Los resultados anteriores muestran que la calificación mínima en el año `2004` fue `9.0` para el actor `Aaron Abrams`.

Por último, podremos calcular la calificación media por año de la siguiente forma.

```
casts_aaron_abrams.groupby(['year']).n.mean()
```

year	n
2003	17.0
2004	14.5
2005	37.5
2006	NaN
2007	16.5
2008	44.0
2009	5.0
2011	13.8
2013	3.0
2015	6.5
2017	1.0
2018	NaN

Según estos resultados, la calificación media en el año `2007` fue `16.5` para el actor `Aaron Abrams`.

Agrupación por campo custom

Supongamos que queremos agrupar los datos en función de las décadas. En este caso necesitamos crear un campo de agrupación personalizado.

```
# Para obtener la década de un año: 1985//10 = 198, 198*10 = 1980
casts_decadas = casts.groupby(casts['year']//10*10).size()
casts_decadas
```

year	0
1910	669
1920	1121
1930	3448
1940	3997
1950	3892
1960	3743
1970	4343
1980	5554
1990	7652
2000	16199
2010	24376
2020	7

Los resultados anteriores muestran el número total de películas en cada década.

Stack/Unstack

`stack()` y `unstack()` son métodos en Pandas que se utilizan para cambiar la estructura de un DataFrame. `stack()` convierte columnas en filas, mientras que `unstack()` convierte filas en columnas. Estos métodos son útiles para transformar los datos en un DataFrame de una forma que sea más fácil de analizar o visualizar.

Por ejemplo, supongamos que tenemos un DataFrame que contiene información sobre el número de ventas de distintos productos en diferentes tiendas.

```
df_ventas = pd.DataFrame([
    {'Tienda 1': 100, 'Tienda 2': 150, 'Tienda 3': 200, 'Tienda 4': 250},
    {'Tienda 1': 50, 'Tienda 2': 100, 'Tienda 3': 150, 'Tienda 4': 200}
])
df_ventas.index = ['Producto 1', 'Producto 2']
df_ventas
```

El DataFrame tiene el siguiente aspecto:

	Tienda 1	Tienda 2	Tienda 3	Tienda 4
Producto 1	100	150	200	250
Producto 2	50	100	150	200

En este caso, cada fila representa un producto, y cada columna representa una tienda. Si quisiéramos analizar las ventas por tienda en lugar de por producto, podríamos utilizar el método `stack()` para convertir las columnas en filas:

```
ventas_por_tienda = df_ventas.stack()
ventas_por_tienda
```

El resultado sería una Serie de Pandas que contiene el número de ventas por tienda, en el siguiente formato:

		0
Producto 1	**Tienda 1**	100
	Tienda 2	150
	Tienda 3	200
	Tienda 4	250
Producto 2	**Tienda 1**	50
	Tienda 2	100
	Tienda 3	150
	Tienda 4	200

dtype: int64

En este caso, cada tienda se convierte en un índice en la Serie, y el número de ventas se convierte en el valor asociado a ese índice.

Por otro lado, si quisiéramos analizar las ventas por producto en lugar de por tienda, podríamos utilizar el método `unstack()` para convertir las filas en columnas:

```
ventas_por_producto = df_ventas.unstack()
ventas_por_producto
```

El resultado sería un DataFrame que contiene el número de ventas por producto, en el siguiente formato:

		0
Tienda 1	Producto 1	100
	Producto 2	50
Tienda 2	Producto 1	150
	Producto 2	100
Tienda 3	Producto 1	200
	Producto 2	150
Tienda 4	Producto 1	250
	Producto 2	200

dtype: int64

En este caso, cada producto se convierte en un índice en el DataFrame, y el número de ventas se convierte en el valor asociado a ese índice.

Ahora que ya comprendemos un poco mejor cómo funcionan los métodos stack() y unstack(), sigamos trabajando con el DataFrame casts que hemos visto en puntos anteriores.

En el siguiente código, los datos se agrupan por década y tipo, es decir, actor y actriz.

```
casts.groupby([casts['year']//10*10, 'type']).size()
```

		0
year	type	
1910	actor	384
	actress	285
1920	actor	710
	actress	411
1930	actor	2628
	actress	820
1940	actor	3014
	actress	983
1950	actor	2877
	actress	1015
1960	actor	2775
	actress	968
1970	actor	3044
	actress	1299
1980	actor	3565
	actress	1989
1990	actor	5108
	actress	2544
2000	actor	10368
	actress	5831
2010	actor	15523
	actress	8853
2020	actor	4
	actress	3

Vamos a generar un gráfico con el número total de actores y actrices en cada década y compararemos.

Una solución a este problema es tomar filas pares e impares por separado y dibujar los datos, lo cual es una operación bastante complicada si los tipos tienen más variedades, como nuevos actores, actrices, etc.

Una solución simple sería utilizar el método `unstack()`, que permite crear un nuevo marco de datos basado en el marco de datos agrupado. Si queremos generar un gráfico basado en actores y actrices, primero debemos agrupar los datos según el tipo como se muestra a continuación.

```
casts_decadas = casts.groupby(['type', casts['year']//10*10]).size()
casts_decadas
```

		0
type	year	
actor	1910	384
	1920	710
	1930	2628
	1940	3014
	1950	2877
	1960	2775
	1970	3044
	1980	3565
	1990	5108
	2000	10368
	2010	15523
	2020	4
actress	1910	285
	1920	411
	1930	820
	1940	983
	1950	1015
	1960	968
	1970	1299
	1980	1989
	1990	2544
	2000	5831
	2010	8853
	2020	3

dtype: int64

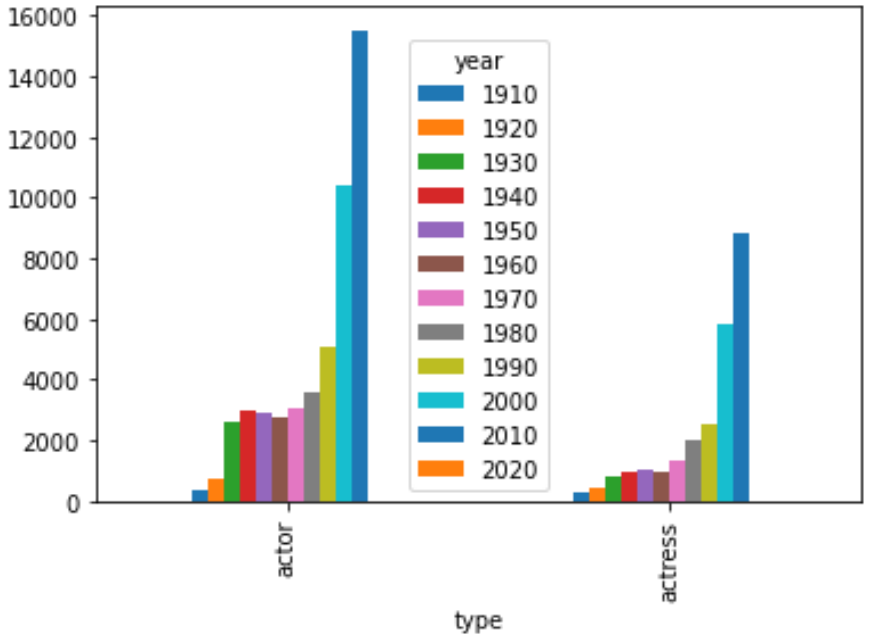
Ahora podemos crear un nuevo DataFrame usando el comando `unstack()`. El comando `unstack()` creará un nuevo marco de datos basado en el índice.

```
casts_decadas.unstack()
```

year	1910	1920	1930	1940	1950	1960	1970	1980	1990	2000	2010	2020
type												
actor	384	710	2628	3014	2877	2775	3044	3565	5108	10368	15523	4
actress	285	411	820	983	1015	968	1299	1989	2544	5831	8853	3

Es el momento de generar el gráfico mediante el método `plot()`, al que le pasaremos como parámetro `kind` el valor `'bar'` para que sea un gráfico de barras.

```
casts_decadas.unstack().plot(kind='bar')
```

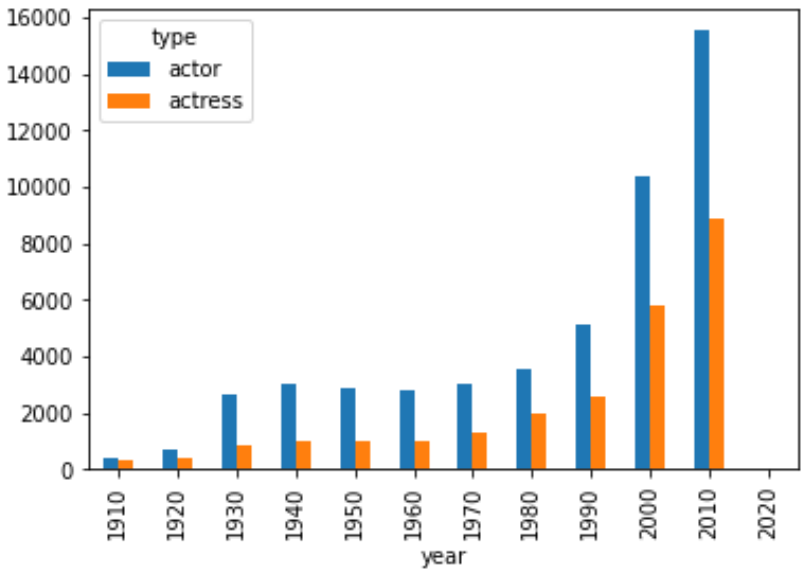


Si queremos representar los datos uno al lado del otro, debemos usar la opción `unstack(0)` como se muestra a continuación (por defecto, se usa `unstack(-1)`).

```
casts_decadas.unstack(0)
```

type	actor	actress
year		
1910	384	285
1920	710	411
1930	2628	820
1940	3014	983
1950	2877	1015
1960	2775	968
1970	3044	1299
1980	3565	1989
1990	5108	2544
2000	10368	5831
2010	15523	8853
2020	4	3

```
casts_decadas.unstack(0).plot(kind='bar')
```



Indexación de los datos

En la sección anterior, vimos algunos usos de `index` para indexar, ordenar y presentar los datos. En esta sección vamos a analizar un poco más en detalle la importancia de la indexación, que es muy importante en Pandas. Se utiliza para organizar los datos y proporcionarnos un acceso más rápido. En esta sección vamos a comparar los tiempos de acceso a los datos con y sin indexación y veremos la diferencia.

Vamos a usar la instrucción `'%%timeit'`, que se usa para medir el tiempo promedio de ejecución de una celda de código después de ejecutarla varias veces. Esto puede ser útil si queremos medir el rendimiento de un algoritmo o código a lo largo del tiempo (*benchmark*). Para usar `'%%timeit'`, simplemente hay que colocarlo al principio de la celda de código y lanzar la ejecución para ver cuánto tiempo tarda en ejecutarse en promedio después de lanzarse varias veces. A `'%%timeit'` le podremos pasar algunos *flags* como `-r` para indicar el número de veces que se va a ejecutar el código en cada iteración del loop, por ejemplo `5`, y `-n` el número de iteraciones que queremos ejecutar, por ejemplo `1000`.

Primero vamos a cargar los datos de nuestro DataFrame `casts` sin indexación alguna de la siguiente manera.

```
casts = pd.read_csv(
    r'datasets/cast.csv',
    encoding='ISO-8859-1'
)
casts
```

	title	year	name	type	character	n
0	Closet Monster	2015	Buffy #1	actor	Buffy 4	31.0
1	Suuri illusioni	1985	Homo $	actor	Guests	22.0
2	Battle of the Sexes	2017	$hutter	actor	Bobby Riggs Fan	10.0
3	Secret in Their Eyes	2015	$hutter	actor	2002 Dodger Fan	NaN
4	Steve Jobs	2015	$hutter	actor	1988 Opera House Patron	NaN
...	...	...	...	...	...	...
74996	Mia fora kai ena... moro	2011	Penelope Anastasopoulou	actress	Popi voulkanizater	11.0
74997	The Magician King	2004	Tiannah Anastassiades	actress	Unicycle Race Attendant	NaN
74998	Festival of Lights	2010	Zoe Anastassiou	actress	Guidance Counselor	20.0
74999	Toxic Tutu	2016	Zoe Anastassiou	actress	Demon of Toxicity	NaN
75000	Fugitive Pieces	2007	Anastassia Anastassopoulou	actress	Laundry Girl	25.0

75001 rows × 6 columns

Si usamos el comando `'%%timeit'` ejecutará el código Python que viene a continuación varias veces y mostrará el tiempo promedio.

```
%%timeit -r 5 -n 1000

casts[casts['title'] == 'Macbeth']
```

```
9.04 ms ± 971 µs per loop (mean ± std. dev. of 5 runs, 1000 loops each)
```

Podemos usar `set_index` para crear un índice para todos los datos. En el código que viene a continuación, se establece el índice en base al campo `title`, por lo tanto, los números de índice se reemplazan por el título de las películas.

```
casts_indexado = casts.set_index(['title'])
casts_indexado
```

title	year	name	type	character	n
Closet Monster	2015	Buffy #1	actor	Buffy 4	31.0
Suuri illusioni	1985	Homo $	actor	Guests	22.0
Battle of the Sexes	2017	$hutter	actor	Bobby Riggs Fan	10.0
Secret in Their Eyes	2015	$hutter	actor	2002 Dodger Fan	NaN
Steve Jobs	2015	$hutter	actor	1988 Opera House Patron	NaN
...	...	...	...	...	...
Mia fora kai ena... moro	2011	Penelope Anastasopoulou	actress	Popi voulkanizater	11.0
The Magician King	2004	Tiannah Anastassiades	actress	Unicycle Race Attendant	NaN
Festival of Lights	2010	Zoe Anastassiou	actress	Guidance Counselor	20.0
Toxic Tutu	2016	Zoe Anastassiou	actress	Demon of Toxicity	NaN
Fugitive Pieces	2007	Anastassia Anastassopoulou	actress	Laundry Girl	25.0

75001 rows × 5 columns

Para hacer un filtro como el anterior, ahora que ya tenemos los datos indexados, debemos usar `loc[]` para realizar las operaciones de una manera mucho más rápida.

```
%%timeit -r 5 -n 1000
```

```
casts_indexado.loc['Macbeth']
```

```
2.71 ms ± 472 µs per loop (mean ± std. dev. of 5 runs, 1000 loops each)
```

Se puede ver que hay una mejora considerable en el rendimiento usando la indexación (de `4.05ms` a `1.37ms`).

Resetear los índices

Podemos restablecer o resetear los índices usando el comando `reset_index`. Veamos de nuevo el DataFrame `casts_indexado`.

```
casts_indexado.head()
```

title	year	name	type	character	n
Closet Monster	2015	Buffy #1	actor	Buffy 4	31.0
Suuri illusioni	1985	Homo $	actor	Guests	22.0
Battle of the Sexes	2017	$hutter	actor	Bobby Riggs Fan	10.0
Secret in Their Eyes	2015	$hutter	actor	2002 Dodger Fan	NaN
Steve Jobs	2015	$hutter	actor	1988 Opera House Patron	NaN

En este momento el DataFrame tiene el índice `title`. Vamos a eliminar el campo `title` como índice con el comando `reset_index`.

```
casts_indexado = casts_indexado.reset_index('title')
casts_indexado.head()
```

	title	year	name	type	character	n
0	Closet Monster	2015	Buffy #1	actor	Buffy 4	31.0
1	Suuri illusioni	1985	Homo $	actor	Guests	22.0
2	Battle of the Sexes	2017	$hutter	actor	Bobby Riggs Fan	10.0
3	Secret in Their Eyes	2015	$hutter	actor	2002 Dodger Fan	NaN
4	Steve Jobs	2015	$hutter	actor	1988 Opera House Patron	NaN

Como se puede observar, el campo `n` ha dejado de ser un índice y vuelve a ser una columna más del DataFrame.

Ejemplo esperanza de vida frente a renta per cápita por países

En este ejemplo vamos a utilizar un DataFrame en el que tenemos la esperanza de vida frente a la renta per cápita por países. Vamos a ver si existe una correlación entre estas dos variables.

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv(
    r'datasets/Info_pais.csv',
    encoding='ISO-8859-1',
    delimiter=';',
    decimal=','
)
```

Ahora pondremos los datos en orden descendente según los valores de la columna `Poblacion` de la siguiente manera:

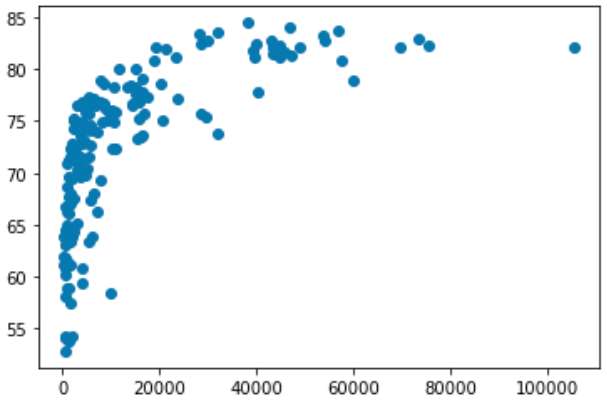
```
df_order = df.sort_values(
    'Poblacion',
    ascending=False
)
df_order
```

	País	Poblacion	Renta per capita	Esperanza de vida
1	China	1421021791	8612	76.7
4	India	1338676785	1980	69.4
0	United States	325084756	59939	78.9
15	Indonesia	264650963	3837	71.5
38	Pakistan	207906209	1467	67.1
...	...	...	...	...
147	Andorra	77001	39128	81.8
68	Luxembourg	59191	105280	82.1
166	Marshall Islands	58058	3517	73.9
162	Vanuatu	28551	3022	70.3
165	Palau	17808	16275	73.7

168 rows × 4 columns

Es el momento de crear el gráfico mediante el método `scatter()`. Representaremos en el eje `x` los datos de la columna `Renta per capita` y en el eje `y` los datos de la columna `Esperanza de vida`.

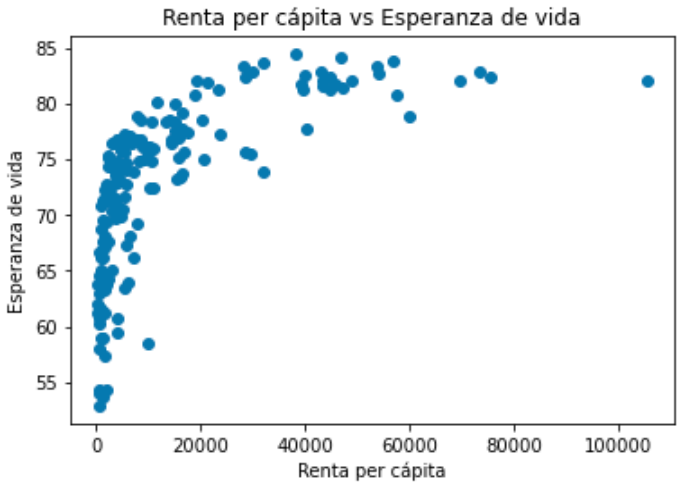
```
plt.scatter(
    df_order['Renta per capita'],
    df_order['Esperanza de vida']
);
```



De momento se aprecia un gráfico que nos da una idea aproximada de cómo se ven los datos. Podemos añadir un título y etiquetas a los ejes `x` e `y` del gráfico de la siguiente manera:

```
plt.scatter(
    df_order['Renta per capita'],
    df_order['Esperanza de vida']
)

plt.title('Renta per cápita vs Esperanza de vida')
plt.xlabel('Renta per cápita')
plt.ylabel('Esperanza de vida');
```



En el gráfico que acabamos de generar los puntos aparecen uniformes. Vamos a configurar el gráfico para que sean proporcionales tanto en tamaño como en color para cada país. Para ello debemos crear una nueva columna llamada por ejemplo `df_order['Poblacion_normalizada']` que normalice frente al máximo de población, por lo que le asignaremos como valores el valor de la columna `Población` dividido entre el valor máximo de esta columna `Población`, de este modo lo estaríamos escalando o normalizando.

```
df_order['Poblacion_normalizada'] = df_order['Poblacion']/max(df_order['Poblacion'])
```

Así, el país que tenga la población más alta quedaría escalado a `1` y el resto de países quedarían normalizados en base a este valor máximo.

Existen grandes diferencias en el número de habitantes entre unos países y otros, por ejemplo China con 1200 millones y otros que podrían tener 50000. Para evitar que un país con esta gran cantidad de habitantes inunde el gráfico es recomendable que en vez de dividir la población de cada país entre el máximo de población, hacer la división del máximo entre `10000`, para no tener un factor tan elevado.

```
df_order['Poblacion_normalizada'] = df_order['Poblacion']/(max(df_order['Poblacion'])/10000)
df_order
```

	País	Poblacion	Renta per capita	Esperanza de vida	Poblacion_normalizada
1	China	1421021791	8612	76.7	10000.000000
4	India	1338676785	1980	69.4	9420.522567
0	United States	325084756	59939	78.9	2287.683117
15	Indonesia	264650963	3837	71.5	1862.399047
38	Pakistan	207906209	1467	67.1	1463.075446
...	...	...	...	...	...
147	Andorra	77001	39128	81.8	0.541871
68	Luxembourg	59191	105280	82.1	0.416538
166	Marshall Islands	58058	3517	73.9	0.408565
162	Vanuatu	28551	3022	70.3	0.200919
165	Palau	17808	16275	73.7	0.125318

168 rows × 5 columns

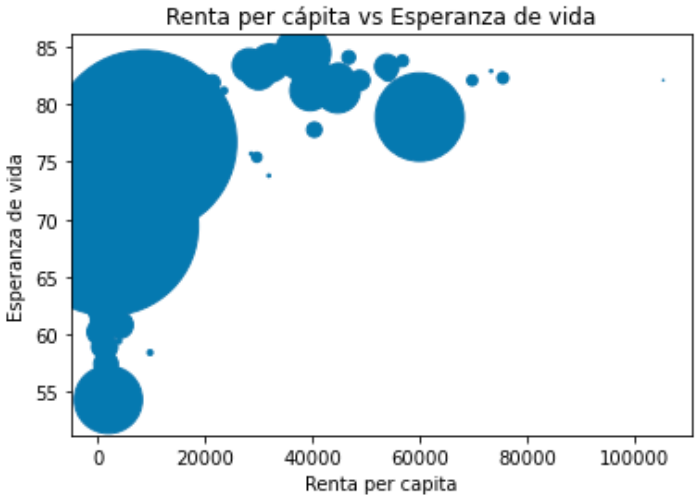
Ahora podremos usar esta nueva columna con los datos escalados para crear una visualización de los datos mucho más potente.

Es el momento de generar un nueva visualización utilizando la nueva columna de datos normalizados que hemos generado, por ejemplo con el siguiente código:

```
plt.scatter(
    df_order['Renta per capita'],
    df_order['Esperanza de vida'],
    s=df_order['Poblacion_normalizada']
)

plt.title('Renta per cápita vs Esperanza de vida')
plt.xlabel('Renta per capita')
plt.ylabel('Esperanza de vida');
```

Para modificar el tamaño hemos utilizado el parámetro `s` (size) y como valor usamos la columna `Poblacion_normalizada`.

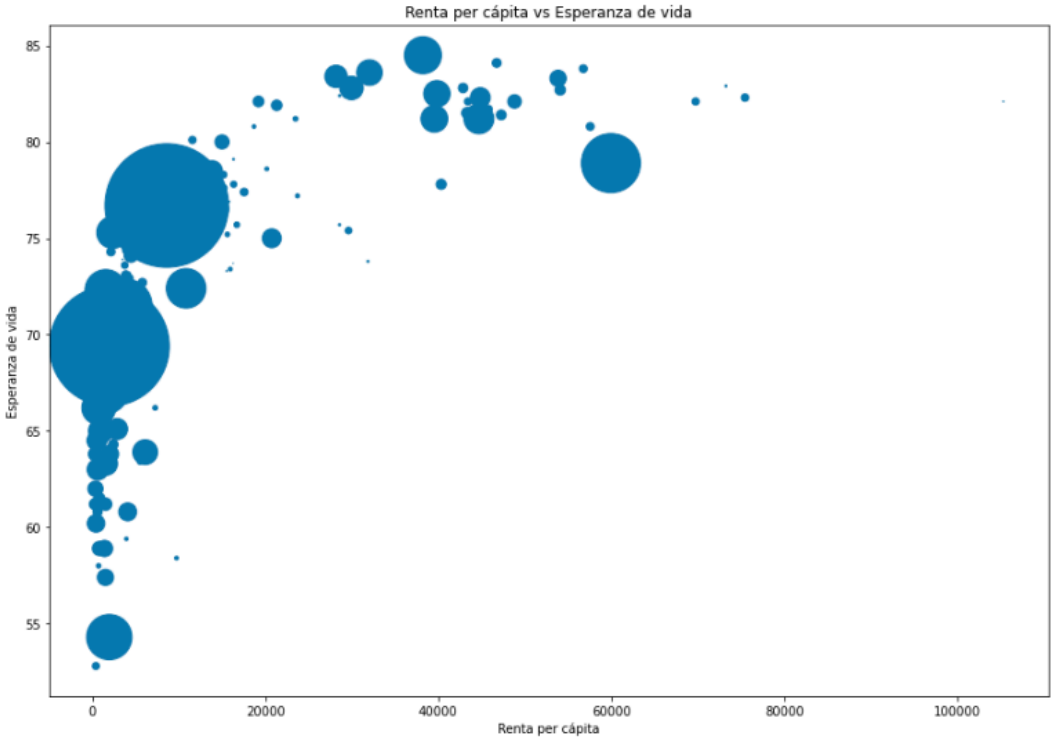


El problema que vemos es que la visualización es muy pequeña, pero podemos mejorar esto añadiendo a nuestro código lo siguiente para aumentar las pulgadas de nuestro gráfico:

```
plt.scatter(
    df_order['Renta per capita'],
    df_order['Esperanza de vida'],
    s=df_order['Poblacion_normalizada']
)

plt.title('Renta per cápita vs Esperanza de vida')
plt.xlabel('Renta per cápita')
plt.ylabel('Esperanza de vida')

fig = plt.gcf()
fig.set_size_inches(14.5, 10);
```



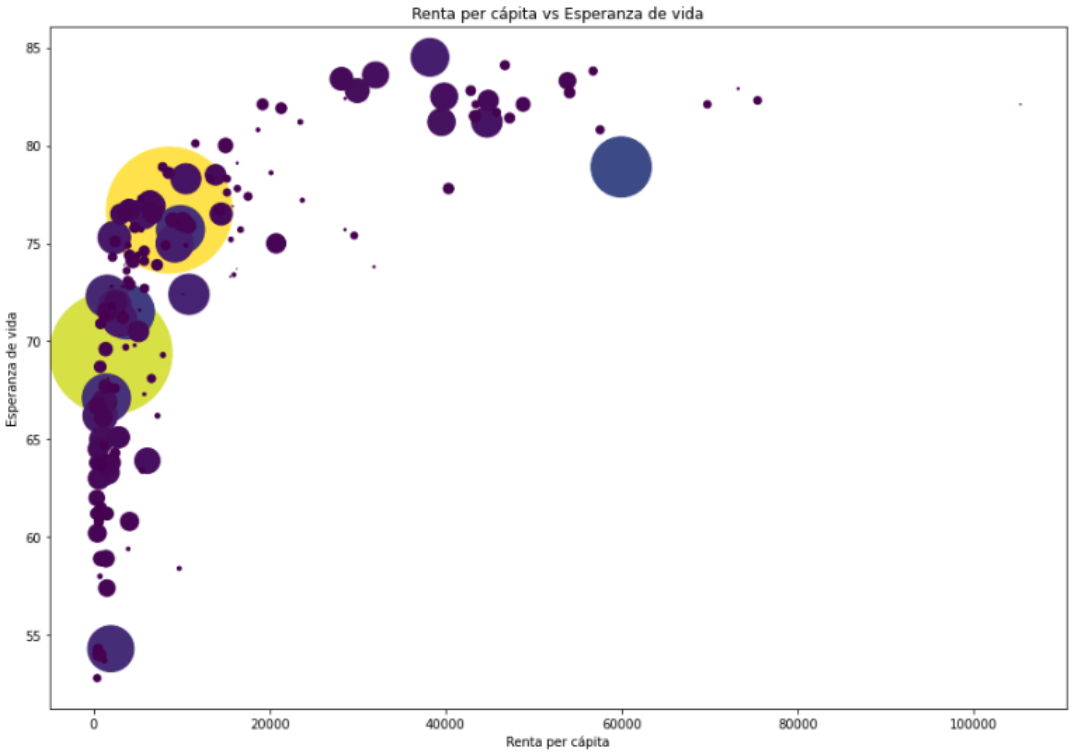
De este modo se ve mucho más grande. Si fuera necesario se pueden cambiar los valores de la función `set_size_inches()` por otros más adecuados para ajustar el tamaño. Si nos fijamos bien, ahora se representa cada burbuja de cada país de un tamaño diferente, dependiendo del valor que tenga en la nueva columna que hemos generado con los datos normalizados.

Ahora vamos a modificar el color. Para ello debemos añadir el atributo `c` (color), a continuación del atributo `s` (size), y como valor vamos a usar de nuevo la columna `Poblacion_normalizada`. Quedaría del siguiente modo:

```
plt.scatter(
    df_order['Renta per capita'],
    df_order['Esperanza de vida'],
    s=df_order['Poblacion_normalizada'],
    c=df_order['Poblacion_normalizada']
)

plt.title('Renta per cápita vs Esperanza de vida')
plt.xlabel('Renta per cápita')
plt.ylabel('Esperanza de vida')

fig = plt.gcf()
fig.set_size_inches(14.5, 10);
```



En esta nueva visualización vemos que ya no solo se representa cada país en un tamaño diferente, sino que también en un color en función de la población.

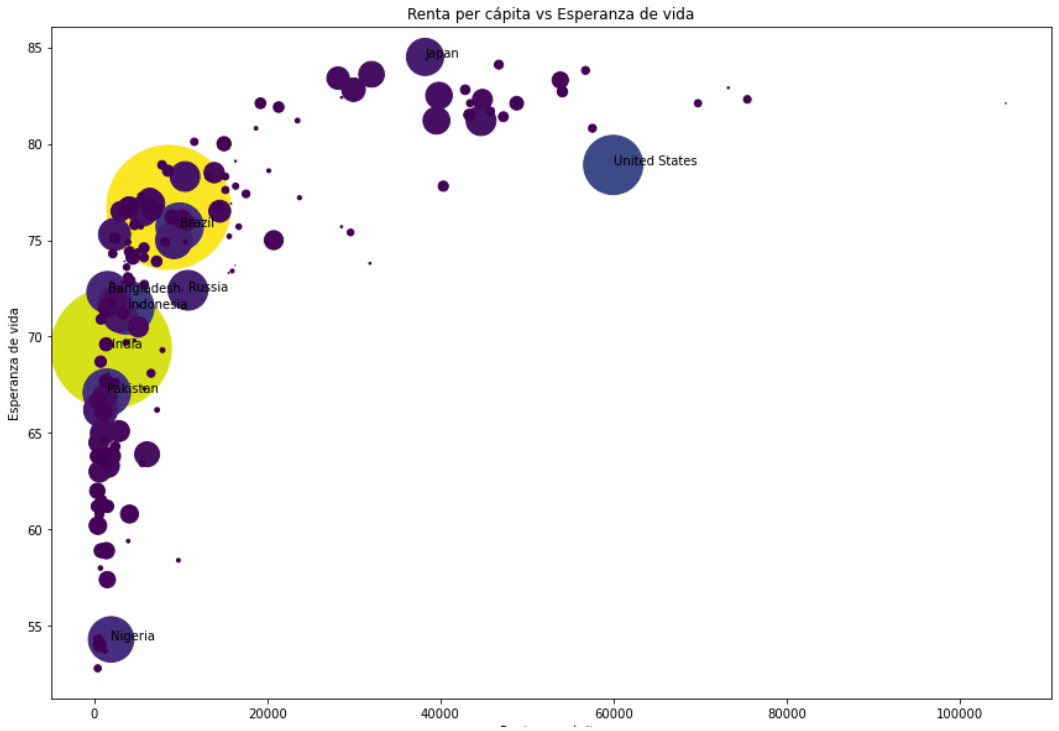
También podremos añadir la etiqueta del nombre del país dentro de cada burbuja utilizando el método `annotate()`, por ejemplo añadiendolo solo a los `10` primeros países con mayor población, pero antes es necesario arreglar los índices utilizando el método `reset_index` y pasándole los parámetros `drop` con valor `True` y `inplace` con valor `True` también. A continuación el código:

```
plt.scatter(
    df_order['Renta per capita'],
    df_order['Esperanza de vida'],
    s=df_order['Poblacion_normalizada'],
    c=df_order['Poblacion_normalizada']
)
plt.title('Renta per cápita vs Esperanza de vida')
plt.xlabel('Renta per cápita')
plt.ylabel('Esperanza de vida')

fig = plt.gcf()
fig.set_size_inches(14.5, 10);

df_order.reset_index(drop=True, inplace=True)
```

```
for i in range(0, 10):
    plt.annotate(
        df_order['País'][i],
        (
            df_order['Renta per capita'][i],
            df_order['Esperanza de vida'][i]
        )
    )
```



Por último, podemos añadir en la generación del scatter una propiedad llamada `alpha` con valor `0.5` para añadir transparencia a los círculos, así podremos ver los datos que se puedan ocultar a sobreponerse unos encima de otros. También añadimos mediante `plt.colorbar()` una barra lateral que indica la escala de colores y qué valores representa.

```
plt.scatter(
    df_order['Renta per capita'],
    df_order['Esperanza de vida'],
    s=df_order['Poblacion_normalizada'],
    c=df_order['Poblacion_normalizada'],
    alpha=0.5
)
plt.title('Renta per cápita vs Esperanza de vida')
plt.xlabel('Renta per cápita')
plt.ylabel('Esperanza de vida')
plt.colorbar()
```

```
fig = plt.gcf()
fig.set_size_inches(14.5, 10);

df_order.reset_index(drop=True, inplace=True)

for i in range(0, 10):
    plt.annotate(
        df_order['País'][i],
        (
            df_order['Renta per capita'][i],
            df_order['Esperanza de vida'][i]
        )
    )
```



De este modo hemos creado una visualización de los datos muy potente y de una manera muy sencilla. La conclusión que podemos sacar de este gráfico es que efectivamente existe una correlación entre la renta per cápita y la esperanza de vida según el país. Podemos ver que conforme la renta per cápita aumenta la esperanza de vida también aumenta. También podemos deducir que el número de población no afecta a la esperanza de vida, ya que en la visualización que hemos creado se pueden ver países con una gran cantidad de población que no están entre los valores más bajos en cuanto a esperanza de vida se refiere.

Escritura en ficheros

Hasta ahora hemos visto ejemplos sobre el uso de Pandas y cómo podemos leer conjuntos de datos desde ficheros, principalmente desde archivos CSV, aunque Pandas permite leer desde muchos otros formatos de archivo.

En este punto veremos cómo poder guardar los datos con los que estamos trabajando, una vez ya los tenemos listos para escribirlos en un fichero. Veamos un ejemplo. Tenemos un archivo CSV llamado `wrong_data.csv`, y este contiene algunas líneas de texto que no nos interesan y que hemos de filtrar.

```
# hey!
a,b,c,d,message
# just wanted to make things more difficult for you
# who reads CSV files with computers, anyway?
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

Podemos leer el archivo de la manera más sencilla que conocemos, y a continuación veamos cómo se ve nuestro DataFrame.

```
df = pd.read_csv(
    r'datasets/wrong_data.csv',
    encoding='ISO-8859-1'
)
df
```

				# hey!
a	b	c	d	message
# just wanted to make things more difficult for you	NaN	NaN	NaN	NaN
# who reads CSV files with computers	anyway?	NaN	NaN	NaN
1	2	3	4	hello
5	6	7	8	world
9	10	11	12	foo

Obviamente se ve mal. Para leer el fichero `wrong_data.csv` sin tener en cuenta las líneas que no nos interesan debemos utilizar el atributo `skiprows` e indicarle el índice de las líneas que queremos omitir. Como en este caso son las líneas número `1`, `3` y `4`, le diremos al atributo `skiprows` que queremos omitir los índices `0`, `2` y `3` de la siguiente manera.

```
df = pd.read_csv(
    r'datasets/wrong_data.csv',
    encoding='ISO-8859-1',
    skiprows=[0, 2, 3]
)
df
```

	a	b	c	d	message
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

Ahora ya se ve bien. Es el momento de guardar el DataFrame en local, en un nuevo archivo al que vamos a llamar `out.csv`.

```
df.to_csv(
    r'datasets/out.csv',
    encoding='ISO-8859-1'
)
```

De este modo habremos generado el siguiente archivo.

```
,a,b,c,d,message
0,1,2,3,4,hello
1,5,6,7,8,world
2,9,10,11,12,foo

```

También podemos guardar el DataFrame sin cabeceras ni índices de la siguiente manera. A este lo llamaremos `out_sin_headers.csv`.

```
df.to_csv(
    r'datasets/out_sin_headers.csv',
    encoding='ISO-8859-1',
    header=False,
    index=False
)
```

Y el archivo resultante se verá de la siguiente manera.

```
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo

```

NumPy

NumPy es una librería de Python enfocada en el cálculo numérico que nos permite realizar operaciones de una manera sencilla y rápida. Su objeto base es un vector de números denominado Array. Es una alternativa a las listas que hemos visto hasta ahora y nos va a permitir realizar una serie de funciones muy potentes. A diferencia de las listas, donde se opera de forma independiente en cada uno de los elementos, en los Arrays las operaciones se van a realizar sobre todo el Array simultáneamente. Por convención la importación de la librería NumPy en nuestro código se realiza de la siguiente manera:

```
import numpy as np
```

Arrays de NumPy vs listas tradicionales

A continuación se muestra una diferencia entre listas y Arrays de NumPy a la hora de realizar una suma de elementos, por ejemplo, tenemos estas dos listas:

```
a = [1, 2, 3]
b = [4, 5, 6]
```

Si sumamos las dos listas obtenemos lo siguiente:

```
a + b
```

```
[1, 2, 3, 4, 5, 6]
```

Como se puede ver, no suma los elementos, solo concatena la lista `b` a continuación de la lista `a`. Veamos cómo se comparta una suma de Arrays con el mismo ejemplo:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b
```

```
array([5, 7, 9])
```

En este caso se han sumado cada uno de los elementos del Array `a` con los elementos del array `b`.

Otra gran diferencia respecto a las listas tradicionales es que en un Array solo se admite un tipo de dato, normalmente numérico. Además NumPy nos va a servir como base de cálculo para otras librerías como Pandas o SciKit Learn.

Ejemplo Índice de Masa Corporal (IMC)

En este ejemplo vamos a calcular el índice de masa corporal sobre los valores `peso` y `altura` de tres personas.

```
altura = [1.7, 1.65, 1.82]
peso = [67, 55, 72]
```

El cálculo que hay que hacer para obtener el IMC es dividir el peso entre el cuadrado de la altura:

```
peso / altura**2
```

Si quisiéramos calcular el IMC mediante listas tradicionales podríamos hacerlo fácilmente usando la función `zip()`, iterar a través de un bucle `for` e ir añadiendo los resultados mediante compresión de listas del siguiente modo:

```
[p / a**2 for p, a in zip(peso, altura)]
```

```
[23.18339100346021, 20.202020202020204, 21.736505252988767]
```

El cálculo está bien hecho, pero de este modo es más complicado, entre otras cosas porque el cálculo se va haciendo secuencialmente elemento a elemento entre las listas, y en este caso no es mucho problema ya que son listas de solo 3 elementos, pero podría ser un problema al trabajar con listas de miles de elementos. Se realizar el mismo cálculo de una manera mucho más eficiente usando los Arrays de NumPy, veamos el ejemplo:

```
np_altura = np.array([1.7, 1.65, 1.82])
np_peso = np.array([67, 55, 72])

imc = np_peso / np_altura**2
imc
```

```
array([23.183391  , 20.2020202 , 21.73650525])
```

Además este tipo de objetos Array de NumPy son iterables del mismo modo que lo son algunas colecciones estándar de Python como las listas, tienen algunas propiedades como los slices que nos permiten navegar dentro del Array y acceder a determinados elementos ubicados en ciertas posiciones.

Otro uso interesante de los Arrays es que podemos evaluar todos los elementos del Array con una simple operación, como por ejemplo saber qué valores son mayores que `21`:

```
imc > 21
```

```
array([ True, False,  True])
```

En este caso el primer resultado es `True` puesto que se cumple que `23.183391` es mayor que `21`, el segundo es `False` porque `20.2020202` no es mayor que `21` y el tercero es `True` puesto que `21.73650525` si es mayor que `21`.

Si quisiéramos obtener solo los elementos del array que cumplen la condición anterior podremos hacerlo de la siguiente manera:

```
imc[imc > 21]
```

```
array([23.183391  , 21.73650525])
```

Arrays de dos dimensiones

Con NumPy también podremos crear un Array de dos dimensiones. En realidad es una matriz de `m` filas por `n` columnas.

$$\mathbb{A} = \begin{bmatrix} a_{(0,0)} & a_{(0,1)} & a_{(0,2)} & \cdots & a_{(0,n)} \\ a_{(1,0)} & a_{(1,1)} & a_{(1,2)} & \cdots & a_{(1,n)} \\ a_{(2,0)} & a_{(2,1)} & a_{(2,2)} & \cdots & a_{(2,n)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{(m,0)} & a_{(m,1)} & a_{(m,2)} & \cdots & a_{(m,n)} \end{bmatrix}$$

La sintaxis para crearla es la siguiente:

```
nombre_array = np.array([[valores_fila_1],
                         [valores_fila_2],
                         [valores_fila_m]])
```

Veamos cómo crear un array bidimensional como este:

$$a = \begin{bmatrix} 2 & 7 & 8 \\ 4 & 8 & 10 \end{bmatrix}$$

```
nombre_array = np.array([[2, 7, 8], [4, 8, 10]])
nombre_array
```

```
array([[ 2,  7,  8],
       [ 4,  8, 10]])
```

Si quisiéramos obtener el valor `10` de nuestro array de dos dimensiones tendríamos que especificar el índice de la fila seguido del índice de la columna, teniendo en cuenta que los índices siempre empiezan por el número `0`. En este ejemplo el valor `10` se encuentra en la fila con índice `1` y la columna con índice `2`.

```
nombre_array[1, 2]
```

```
10
```

Ahora supongamos que queremos obtener todas las filas pero solo los valores de las columnas primera y segunda. En este caso tendríamos que especificar en primer lugar que queremos todas las filas mediante los dos puntos `:`, y a continuación un slice `0:2` para indicar solo las columnas desde el índice `0` hasta el `1`, ya que en un slice el número que se indica al final no se muestra, es donde se para.

```
nombre_array[:, 0:2]
```

```
array([[2, 7],
       [4, 8]])
```

Cálculo estadístico

Para realizar cálculo estadístico NumPy nos ofrece una gran variedad de funciones muy útiles y potentes. En esta sección veremos algunas de ellas que nos pueden servir para solucionar algunos cálculos que hemos visto en puntos anteriores de una manera mucho más sencilla y rápida.

Por ejemplo, supongamos que tenemos el siguiente Array con varios registros de temperaturas de una ciudad:

```
temperaturas = np.array([12, 13.5, 13, 14, 13.2, 14.8, 15, 15.16, 16, 16.2,
15.7, 17, 17.2, 16.8, 14, 14.2, 14.7, 16, 17.5])
```

Podríamos calcular la media o promedio usando la función `mean(array)` de la siguiente manera:

```
np.mean(temperaturas)
```

```
15.050526315789472
```

La mediana usando la función `median(array)`:

```
np.median(temperaturas)
```

```
15.0
```

Podremos obtener el valor mínimo con la función `min(array)`.

```
np.min(temperaturas)
```

```
12.0
```

Y el valor máximo con la función `max(array)`.

```
np.max(temperaturas)
```

```
17.5
```

Para calcular la varianza podremos usar la función `var(array)` de la siguiente manera:

```
np.var(temperaturas)
```

```
2.2893207756232683
```

La desviación estándar podremos obtenerla usando la función `std(array)` de la siguiente manera:

```
np.std(temperaturas)
```

```
1.5130501563475245
```

También tenemos la función `percentile(array, k)` a la que tendremos que pasarle como argumentos el array y también el percentil que queremos obtener, por ejemplo el `90%`:

```
np.percentile(temperaturas, 90)
```

```
17.04
```

Esto lo que quiere decir es que en el array de temperaturas no se supera la temperatura `17.04` en el `90%` de los casos, o dicho de otro modo, solo el `10%` de los datos supera la temperatura de `17.04`.

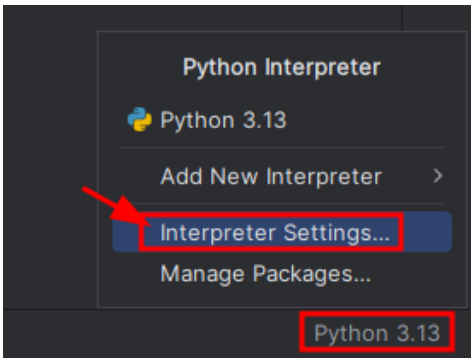
Servicio web

Nota: *Esta sección de la documentación se ha de realizar en **PyCharm***.

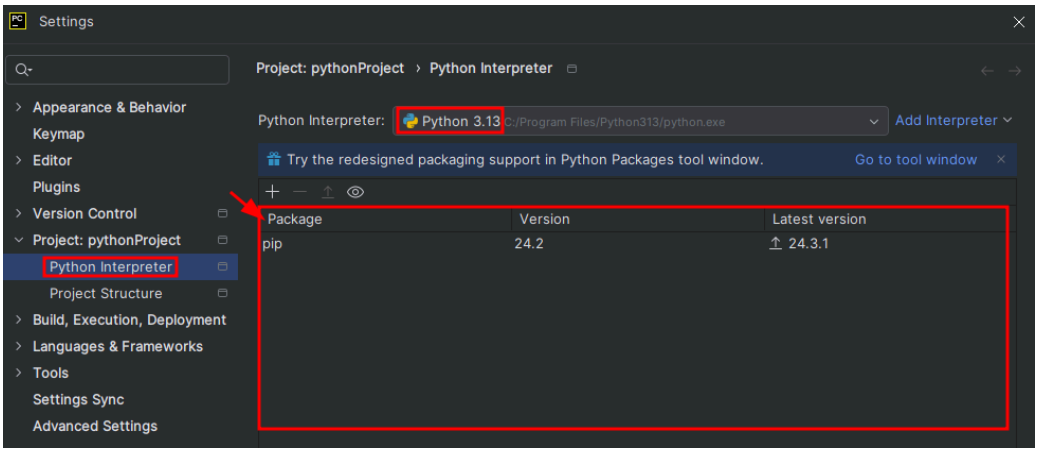
Instalación de librerías necesarias

Se puede crear un servicio web con Python de varias maneras diferentes, pero para este ejemplo usaremos un método bastante sencillo. Únicamente necesitamos instalar dos librerías: **Flask** y **gunicorn**.

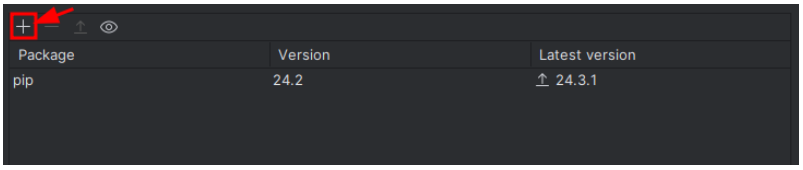
1. Para instalar Flask desde PyCharm hacemos click sobre "*Python 3.13*" (o la versión que tengas), que aparece en la esquina inferior derecha de la ventana.



2. En la ventana que emerge podremos ver en la opción "*Python Interpreter*" la versión que estamos utilizando en este proyecto, que en este caso es "*Python 3.13*", y en la ventana de abajo se muestran listados los paquetes que actualmente tenemos instalados y sus versiones, es posible que aparezca únicamente el paquete "pip".

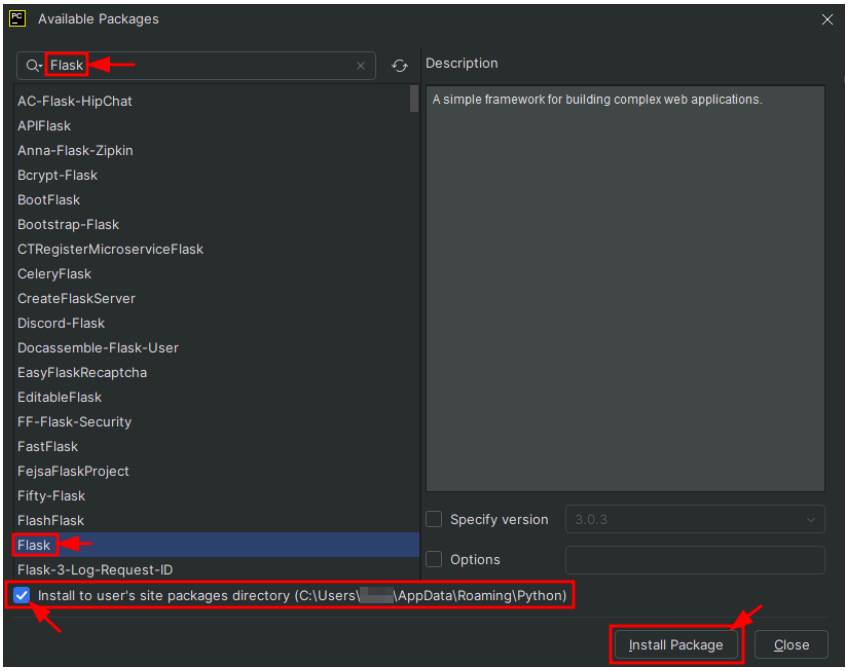


3. Para instalar un paquete nuevo hemos de hacer click sobre el botón "+" tal y como se muestra en la siguiente imagen:

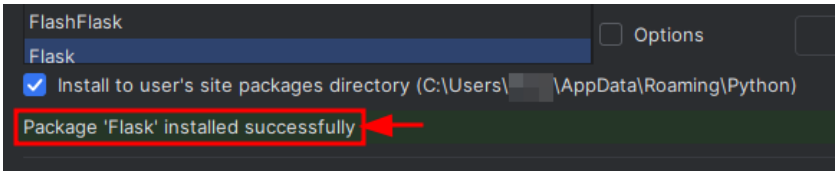


4. En la barra de búsqueda escribimos "*Flask*", y nos aparecerán varios paquetes en los que coincide el literal que buscamos. Hemos de

seleccionar el primero, el que se llama solo "*Flask*" y pulsamos el botón "*Install Package*".

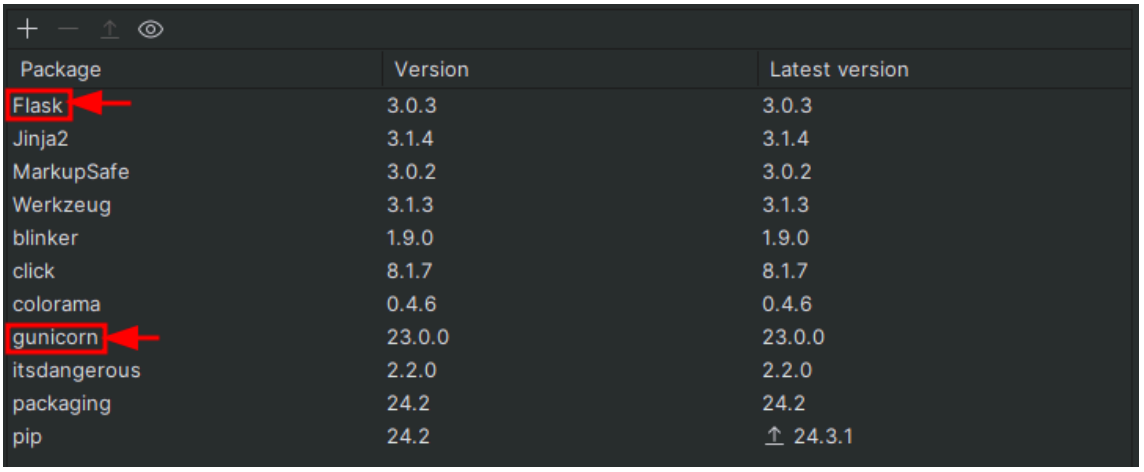


5. Cuando se termine de instalar deberemos ver un mensaje que dice "*Package 'Flask' installed successfully*" tal y como se muestra en la siguiente imagen. Podemos cerrar esta ventana:



6. Por último, hay que instalar el paquete "*gunicorn*" siguiendo los mismos pasos que hemos realizado en la instalación del paquete "*Flask*".

Una vez instalados, la lista de paquetes aparecerá con muchos más paquetes instalados, tal y como se muestra en la imagen, estos otros paquetes son dependencias que se auto instalan.



Package	Version	Latest version
Flask	3.0.3	3.0.3
Jinja2	3.1.4	3.1.4
MarkupSafe	3.0.2	3.0.2
Werkzeug	3.1.3	3.1.3
blinker	1.9.0	1.9.0
click	8.1.7	8.1.7
colorama	0.4.6	0.4.6
gunicorn	23.0.0	23.0.0
itsdangerous	2.2.0	2.2.0
packaging	24.2	24.2
pip	24.2	24.3.1

Desarrollo de una API Rest

Primero importamos las librerías necesarias:

```
import json

from werkzeug.wrappers import Response
from flask import Flask
```

A continuación instanciamos un objeto de tipo `Flask` y lo almacenamos en una variable llamada `app`.

```
app = Flask(__name__)
```

Esta será la base de nuestra aplicación web.

Ahora añadiremos una función llamada `home()` con el siguiente código:

```
@app.route('/')
def home():
  return '<h3>Hola mundo!</h3>'
```

El código `@app.route('/')` que está encima de la firma de la función `home` se llama decorador, y en este caso lo utilizaremos para indicar a la aplicación web que esta función se invocará únicamente cuando se haga una petición desde un navegador a la URL de nuestra aplicación compuesta por un *hostname:puerto*, sin ningún path adicional (por ejemplo

http://localhost:9000/). En el cuerpo de la función únicamente tenemos el retorno de un string que realmente es un código HTML con una cabecera de nivel 3 (`<h3>`) y el texto "`Hola mundo!`".

Después añadiremos otra función similar, pero con algún cambio en el decorador y un retorno un poco más complejo. A esta función la llamaremos `get_alumnos()`:

```
@app.route('/alumnos')
def get_alumnos():
  data = {
    'centro': 'Universidad Complutense de Madrid',
    'alumnos': [
      {'id': '00', 'matrícula': '1234QSPM', 'nombre': 'Mari Carmen', 'apellidos': 'García', 'clase': 'A'},
      {'id': '01', 'matrícula': '1234NDFK', 'nombre': 'Antonio', 'apellidos': 'Rodríguez', 'clase': 'A'},
      {'id': '02', 'matrícula': '1234KWSH', 'nombre': 'María', 'apellidos': 'González', 'clase': 'B'},
      {'id': '03', 'matrícula': '1234ZMLD', 'nombre': 'Manuel', 'apellidos': 'Fernández', 'clase': 'A'},
      {'id': '04', 'matrícula': '1234POWM', 'nombre': 'Carmen', 'apellidos': 'López', 'clase': 'A'},
      {'id': '05', 'matrícula': '1234LJBF', 'nombre': 'Jose', 'apellidos': 'Martínez', 'clase': 'B'},
      {'id': '06', 'matrícula': '1234GQKD', 'nombre': 'Ana María', 'apellidos': 'Sánchez', 'clase': 'B'},
      {'id': '07', 'matrícula': '1234KJEM', 'nombre': 'Francisco', 'apellidos': 'Pérez', 'clase': 'A'},
      {'id': '08', 'matrícula': '1234TTMK', 'nombre': 'María Pilar', 'apellidos': 'Gómez', 'clase': 'A'},
      {'id': '09', 'matrícula': '1234EFRW', 'nombre': 'David', 'apellidos': 'Martín', 'clase': 'A'},
      {'id': '10', 'matrícula': '1234LWJB', 'nombre': 'Laura', 'apellidos': 'Jiménez', 'clase': 'B'},
      {'id': '11', 'matrícula': '1234GFDV', 'nombre': 'Juan', 'apellidos': 'Hernández', 'clase': 'B'},
      {'id': '12', 'matrícula': '1234MKWX', 'nombre': 'Josefa', 'apellidos': 'Ruíz', 'clase': 'B'},
      {'id': '13', 'matrícula': '1234NSHC', 'nombre': 'Javier', 'apellidos': 'Díaz', 'clase': 'A'},
      {'id': '14', 'matrícula': '1234PCER', 'nombre': 'Isabel', 'apellidos': 'Moreno', 'clase': 'B'},
      {'id': '15', 'matrícula': '1234AHIV', 'nombre': 'Jose Antonio', 'apellidos': 'Muñoz', 'clase': 'A'},
      {'id': '16', 'matrícula': '1234MCTM', 'nombre': 'María Dolores', 'apellidos': 'Álvarez','clase': 'B'}
    ]
  }
  return Response(
    json.dumps(data, ensure_ascii=False),
    mimetype='application/json'
)
```

En este caso el decorador es `@app.route('/alumnos')`, quiere decir que esta función `get_alumnos()` se invocará únicamente cuando se haga una petición desde un navegador a la URL de nuestra aplicación compuesta por un *hostname:puerto/alumnos* (por ejemplo http://localhost:9000/alumnos). En el cuerpo de la función tenemos por un lado una variable llamada `data` cuyo valor es una lista de diccionarios, y cada uno de estos diccionarios contiene los datos de alumnos (ficticios, son datos random), y por otro lado el retorno de esta colección de datos en formato JSON. El atributo `ensure_ascii` con valor `False` lo usamos para que nos respete los caracteres especiales como las vocales con acentos, las eñes, etc.

Por último añadiremos al final de nuestro programa el siguiente código:

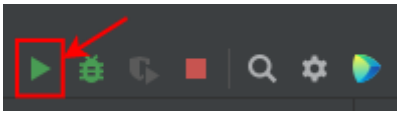
```
if __name__ == '__main__':
  from werkzeug.serving import run_simple

  run_simple('localhost', 9000, app)
```

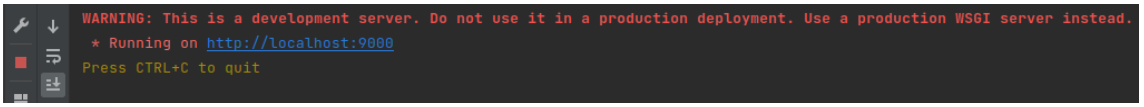
Dentro de un contexto `'__main__'` lo primero que hacemos es importar el módulo `run_simple` del paquete `werkzeug.serving`. Finalmente, lo invocamos pasándole como argumentos `localhost` como nombre del servicio, `9000` como puerto en el que corre el servicio web en nuestro PC y finalmente nuestro objeto `app`.

Extracción de datos de una API

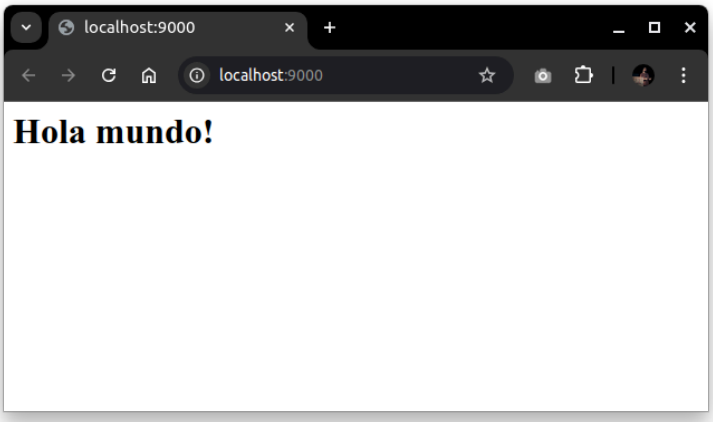
Ahora es el momento de ejecutar el código del servicio web que hemos desarrollado en el punto anterior y ver cómo funciona.



En la terminal de PyCharm aparecerá un mensaje como el siguiente:

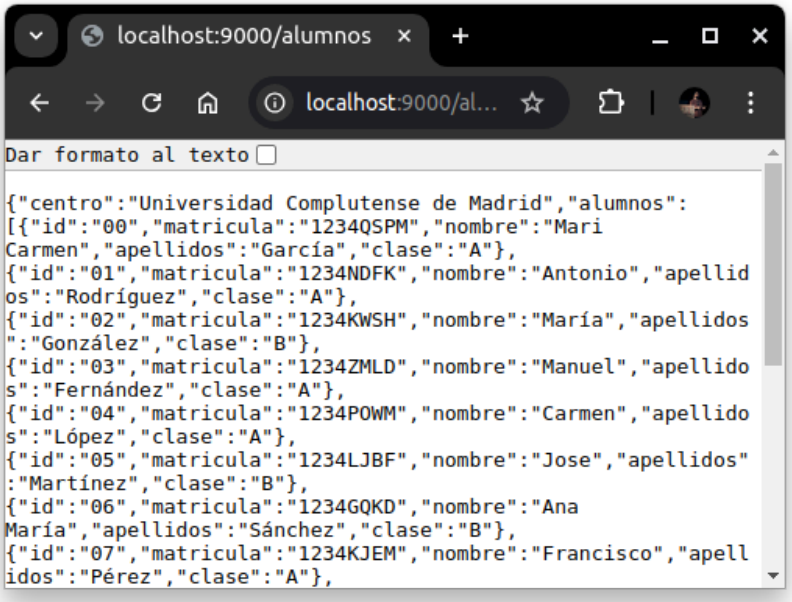


Ahora el servicio permanece iniciado y no debemos pararlo. Podemos hacer una petición web desde un navegador web cualquiera a la URL de nuestro servicio: http://localhost:9000/.

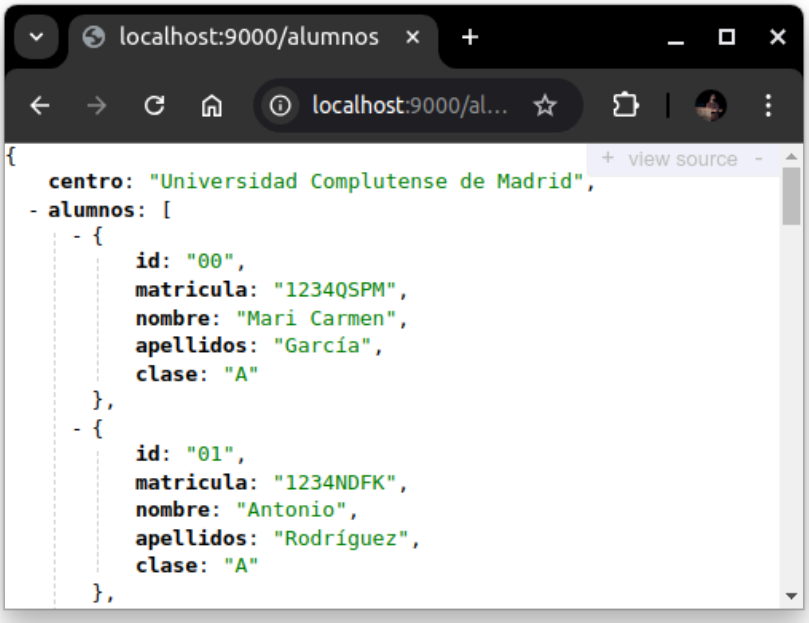


El navegador web nos mostrará una página web en la que podremos leer el mensaje "**Hola mundo!**" que definimos en el código de nuestra aplicación web en la función `home()`.

Ahora hacemos otra petición web a esta misma URL pero añadiendo el path `/alumnos` después del puerto: http://localhost:9000/alumnos



Podemos observar que esta parte o *endpoint* de la API nos devuelve toda la información de los alumnos en formato JSON. También hemos observado que nuestra API devuelve la información en una sola línea muy larga, sin identar o tabular. Esto es bastante habitual, pues las APIs normalmente suelen devolver gran cantidad de información y no suele ser procesada por un humano, por lo que no necesita añadir caracteres adicionales como saltos de línea o tabuladores. Lo normal es que la respuesta se procese con algún programa o proceso automático. Pero si queremos verlo más legible podemos instalar en nuestro navegador algún plugin como JSONVUE que nos mostrará la información de este tipo de respuestas JSON de una manera más presentable, por ejemplo:



Ahora veamos cómo podemos obtener o procesar esta misma petición web al *endpoint* "`/alumnos`" de nuestra API desde Python. Bastaría con emplear el siguiente código:

```
import json
import requests

url = 'http://localhost:9000/alumnos'

try:
  response = requests.get(url)
import json
import requests

url = 'http://localhost:9000/alumnos'

try:
  response = requests.get(url)

  print(f'Respuesta obtenida en {response.elapsed.total_seconds()} segundos.')

  response_dict = json.loads(response.text)

  for alumno in response_dict['alumnos']:
    print(alumno)
except Exception as err:
  print(f'Ha habido un error: {err}')
```

```
  print(f'Respuesta obtenida en {response.elapsed.total_seconds()} segundos.')

  response_dict = json.loads(response.text)

  for alumno in response_dict['alumnos']:
    print(alumno)
except Exception as err:
  print(f'Ha habido un error: {err}')
```

Al inicio importamos las librerías `json` y `requests`. Luego establecemos una variable url con la dirección web de nuestra API y el endpoint "`/alumnos`". Después, en el contexto de un `try`, ejecutaremos nuestra petición web a la URL establecida anteriormente y almacenaremos la respuesta en la variable `response`. Una vez obtenida la respuesta, esta tiene un montón de metadatos, y uno de ellos es el tiempo que ha tardado en ejecutarse y ser devuelta. Este dato lo podremos obtener mediante el método `elapsed.total_seconds()` de nuestra respuesta almacenada en la variable `response`. Imprimimos este dato para verlo por pantalla, son microsegundos (millonésima parte de un segundo).

Luego convertimos la respuesta JSON a un diccionario Python con `json.loads()` . Recordemos que la respuesta es un JSON de dos campos (`centro` y `alumnos`). Finalmente iteramos por cada elemento del campo "`alumnos`" para imprimir por pantalla cada uno de los alumnos.

Si todo ha ido bien veremos una salida como la siguiente por la terminal de PyCharm:



Y si algo fue mal veremos el error por pantalla también mediante la sentencia `except` que sucede al método `try`. Por ejemplo, si la API no estuviera disponible por que hemos parado el servicio:

